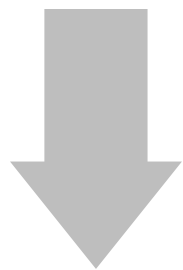


- 有这样一类问题：它有 n 个输入，而它的解就是这 n 个输入的某个子集，这些子集必须满足某些事先给定的条件。
- 约束条件：必须满足的条件
- 可行解：满足约束条件的子集
- 目标函数：为了衡量可行解的优劣，预先给定衡量标准，以函数形式给出
- 最优解：使目标函数取极值(极大值或极小值)的可行解

提出贪心策略



证明策略正确

观察问题特征，构造贪心选择

假设最优方案，通过替换证明

- 假定只能在一台机器上处理 n 个作业, 每个作业均可在单位时间内完成; 又假定每个作业 i 都有一个截止期限 $d_i > 0$ (d_i 是整数), 当且仅当作业 i 在它的期限截止之前被完成时获得 $p_i > 0$ 的效益。
- 该问题的一个可行解
 - 是这 n 个作业的一个子集合 J : J 中的每个作业都能在各自的截止期限之前完成, 可行解的效益值是 J 中这些作业的效益之和 $\sum p_j$
- 该问题的一个最优解
 - 具有最大效益值的可行解

限制条件

目标函数

- 输入:
 - 效益($p_1, p_2, p_3, p_4, p_5, p_6, p_7$) =
(3, 1, 3, 4, 5, 2, 10)
 - 期限($d_1, d_2, d_3, d_4, d_5, d_6, d_7$) =
(1, 2, 2, 4, 4, 4, 4)
- 输出: 总效益最大的可行解
- 寻找最优量度标准
 - 以目标函数 $\sum p_i$ 作为量度标准, 将各作业按效益 p_i 降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$

当前作业一旦满足约束条件,
问题就能获得的最大效益增量

```
J ← {1}
for i ← 2 to n do
    if (JU{i}的所有作业都能在它们的截止
        期限前完成)
        then J ← JU{i}
    end
end
```

- 正确策略：效益优先

最优解 I

贪心解 J

$I = J?$

J 是最优解!

$I \subset J?$

I 是最优解, 不可能!

$I \supset J?$

由贪心策略, 不可能!

存在 $a \in J \setminus I, b \in I \setminus J$

设 a 为 $J \setminus I$ 中效益最大的作业,
则 $p_a \geq \max_{b \in I \setminus J} p_b$

- 正确策略：效益优先

最优解 I

依次检查并替换

重新排列相同作业
不同作业，替换

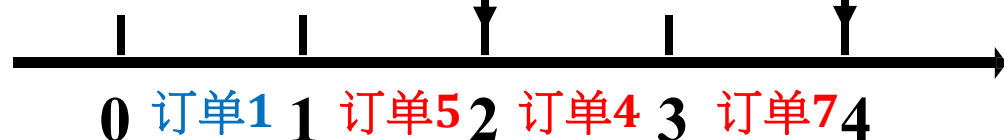
贪心解 J

由 $p_a \geq p_b$ ，替换后的解 I' 仍是一个最优解！
但 I' 相比于 I 少一个与 J 不同的作业！
重复直至 $I' = J$

订单4	4元
订单5	5元
订单7	10元

订单 b

订单3	3元
-----	----



订单 a

订单1	3元
-----	----

订单4	4元
订单5	5元
订单7	10元



如何判断可行策略？

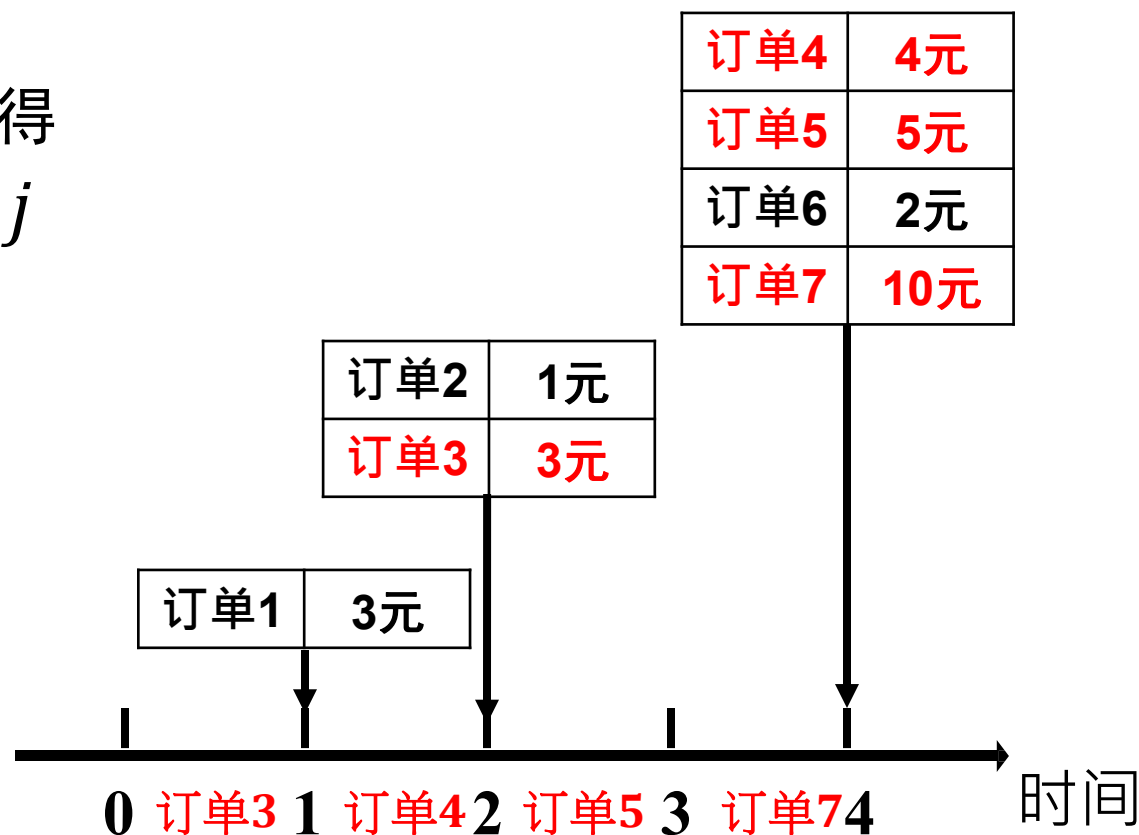
- 输入：作业期限($d_1, d_2, \dots, d_k$)
- 输出：是否一个排列 $i_1, i_2, i_3, \dots, i_k$ ，使得
$$\forall j = 1, 2, \dots, k, d_{i_j} \geq j$$

- 贪心策略：

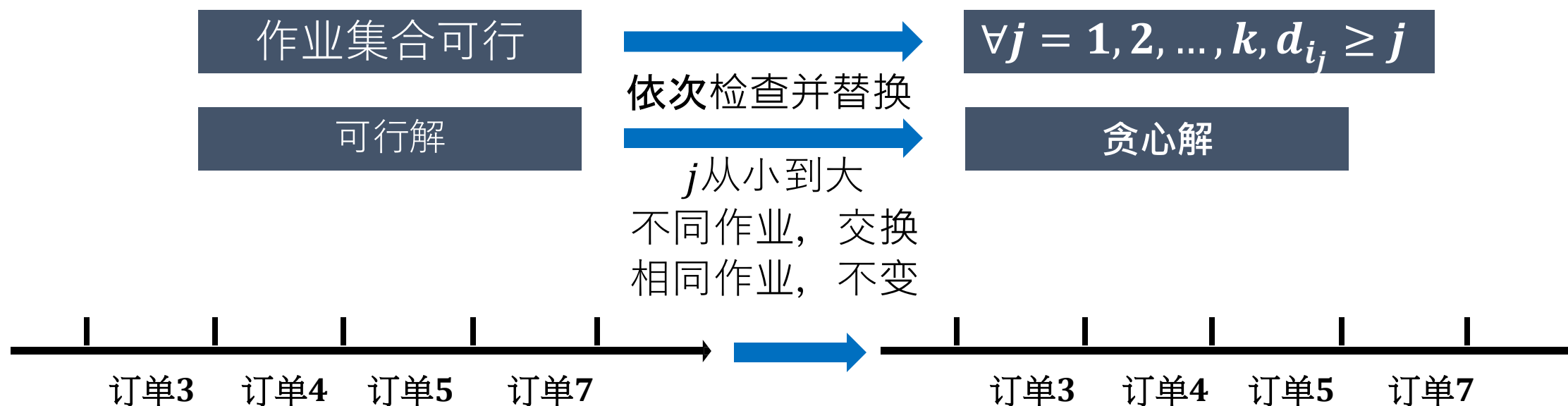
- 按**期限**升序对作业进行排列

	订单3	订单4	订单5	订单7
序号	1	2	3	4
期限	2	4	4	4

- 可行当且仅当**期限** $\geq$ **序号**



- 贪心策略：
 - 按**期限**升序对作业进行排列




```
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
```

- 若还没给作业 i 分配处理时间, 则分给它时间片 $[\alpha-1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则 i 不计入 J 中。
- 旨在: 去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

思考: 如何找到 α ?

时间换空间

定义一个数组记录期限 d 当前可用的最大空时间片?

- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha-1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在: 去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$
- 定义一维数组*F*, *F*(*d*)表示时间期限为*d*的作业当前可用的最大可用空时间片

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



期限	1	2	3	4
F	1	2	3	4



期限	1	2	3	4
F	1	2	3	3



期限	1	2	3	4
F	1	2	2	2

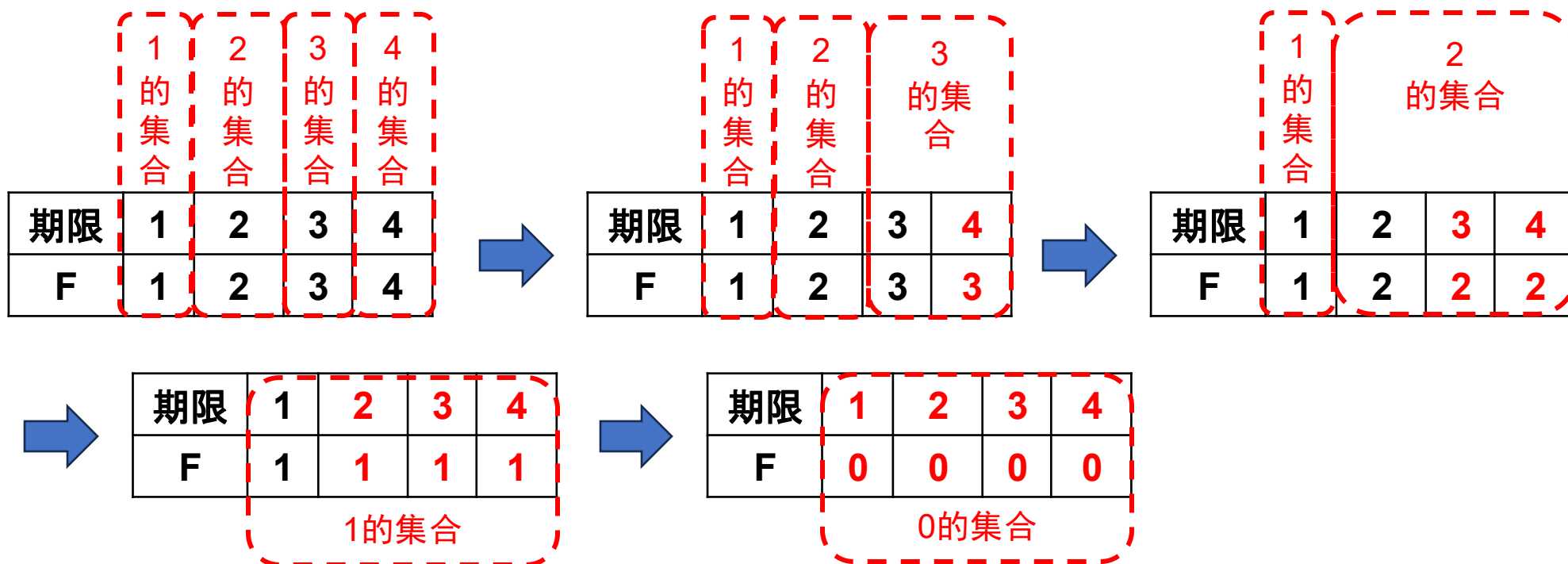


期限	1	2	3	4
F	1	1	1	1



期限	1	2	3	4
F	0	0	0	0

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。
- 直接维护F，复杂度依然为 $O(n)$ ，需要更高效的数据结构！
- 把F看作多个最大可用空时间的不相交集！



```
// 找最优解 $J=J(1),\dots,J(k)$ , 已知 $p_1 \geq p_2 \dots \geq p_n$ ,  $b=\min\{n, \max\{d_j\}\}$ 
for i  $\leftarrow$  0 to b do
    F(i)  $\leftarrow$  i;
    Create-Set(i)
end
k $\leftarrow$  0
for i  $\leftarrow$  1 to n do
    j $\leftarrow$  Find-Set(min(n, D(i)))
    if F(j)  $\neq$  0 then
        k  $\leftarrow$  k+1; J(k)  $\leftarrow$  i
        l $\leftarrow$  Find-Set(F(j)-1)
        Union-Set(l, j)
        F(j)  $\leftarrow$  F(l)
    end
end
End FJS
```

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7)=(4, 2, 4, 3, 4, 8, 3)$

```
for i ← 0 to b do  
  F(i) ← i;  
  Create-Set(i)  
end
```

考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

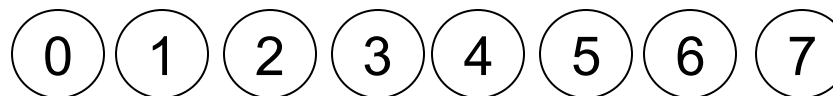
无

0 1 2 3 4 5 6 7

不相交集合

J

ϕ



- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 4$
 $l = 3$
 $\text{Union-Set}(3, 4)$
 $F(4) \cup F(3)$

考虑
作业

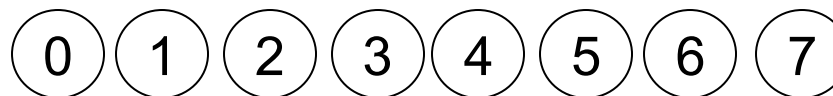
不相交集

J

无

F(0)(1)(2)(3)(4)(5)(6)(7)

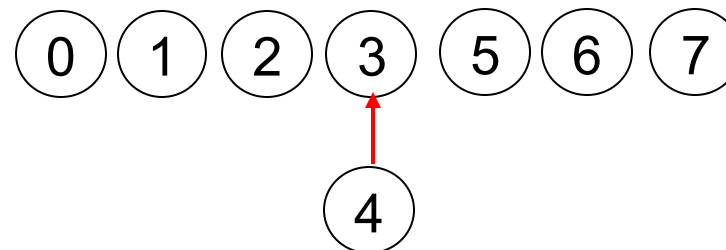
0 1 2 3 4 5 6 7



ϕ

1

0 1 2 3 3 5 6 7



{1}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 2, j = 2$
 $l = 1$
 $\text{Union-Set}(1, 2)$
 $F(2) \cup F(1)$

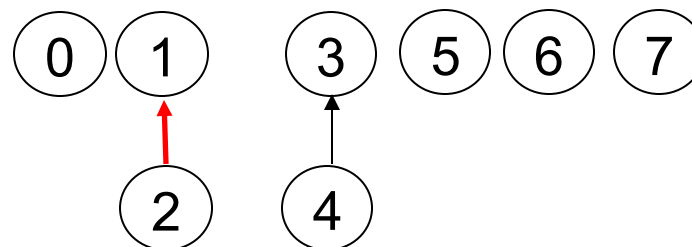
考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

2

0 1 **1** 3 3 5 6 7

不相交集集合



J

{1,2}

算法示例

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 3$
 $l = 1$
 $\text{Union-Set}(1, 3)$
 $F(3) \cup F(1)$

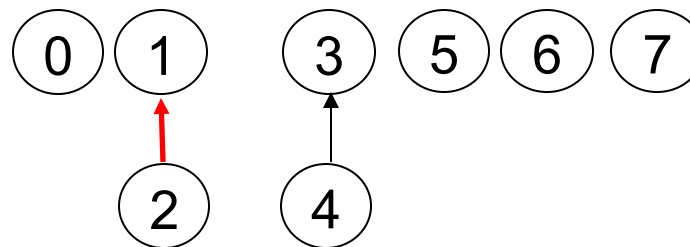
考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

2

0 1 **1** 3 3 5 6 7

不相交集合并

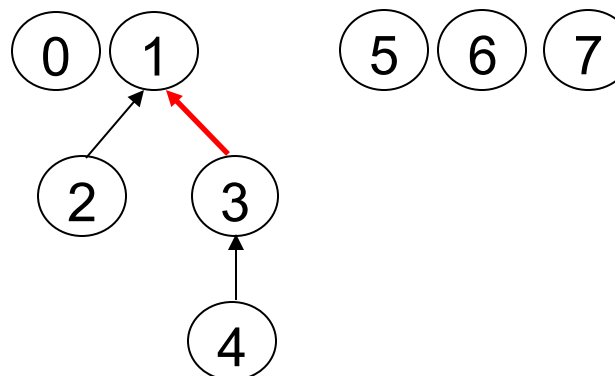


J

{1,2}

3

0 1 1 **1** 3 5 6 7



{1,2,3}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

考虑
作业

4

$F(0)(1)(2)(3)(4)(5)(6)(7)$

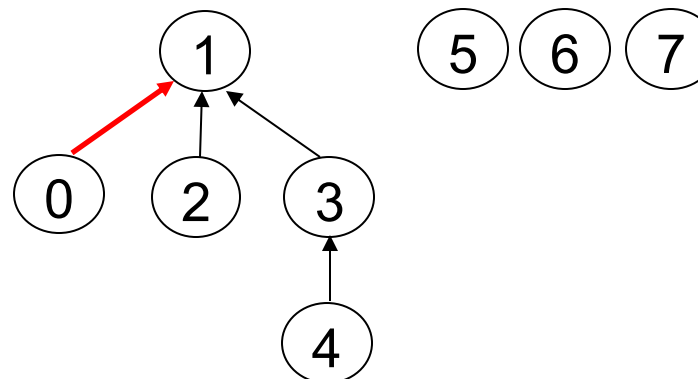
0 0 1 1 3 5 6 7

思考：为何如此合并？

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

不相交集集合



$D(i) = 3, j = 1$
 $l = 0$
 Union-Set(0, 1)
 $F(1) \cup F(0)$

J

{1,2,3,4}

算法示例

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 1$
 $F(j) = 0$

考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

4

0 0 1 1 3 5 6 7

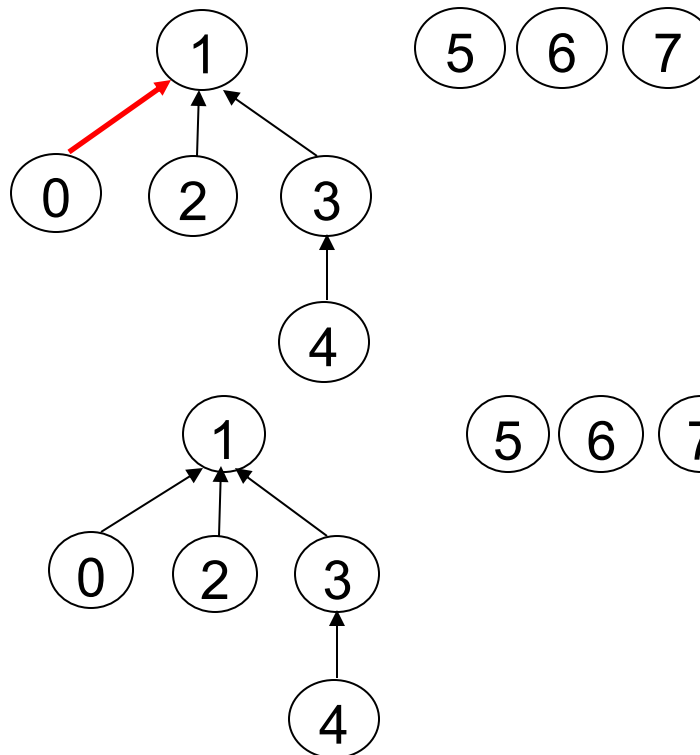
思考：为何如此合并？

5
(舍弃)

0 0 1 1 3 5 6 7

不相交集

J



{1,2,3,4}

{1,2,3,4}

算法示例

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 7, j = 7$
 $l = 6$
 $\text{Union-Set}(6, 7)$
 $F(7) \cup F(6)$

考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

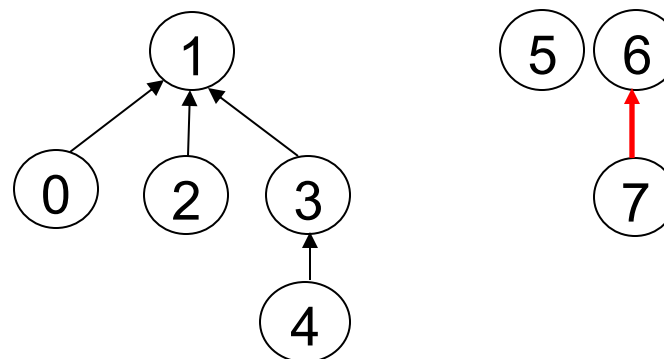
6

0 0 1 1 3 5 6 6

不相交集

J

{1,2,3,4,6}



算法示例

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7)=(35,30,25,20,15,10,5)$
- $(d_1, d_2, \dots, d_7)=(4,2,4,3,4,8,3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k+1; J(k) ← i
    l ← Find-Set(F(j)-1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 7, j = 7$
 $l = 6$
 $\text{Union-Set}(6, 7)$
 $F(7) \cup F(6)$

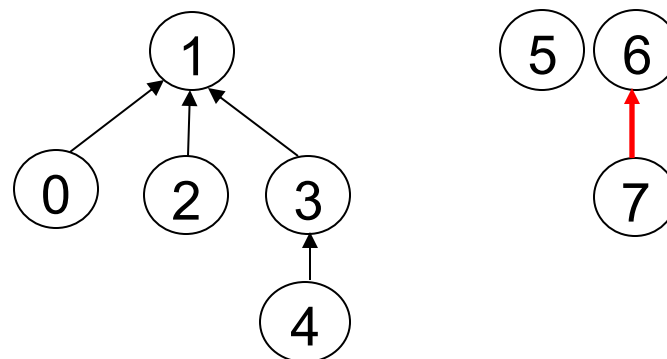
考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

6

0 0 1 1 3 5 6 6

不相交集



J

{1,2,3,4,6}

7

0 0 1 1 3 5 6 6

(舍弃)

同上

最优解 $J = \{1,2,3,4,6\}$, 处理次序4,2,3,1,6, 效益值120



School of Artificial Intelligence
Jilin University

第五章 动态规划

吉林大学人工智能学院

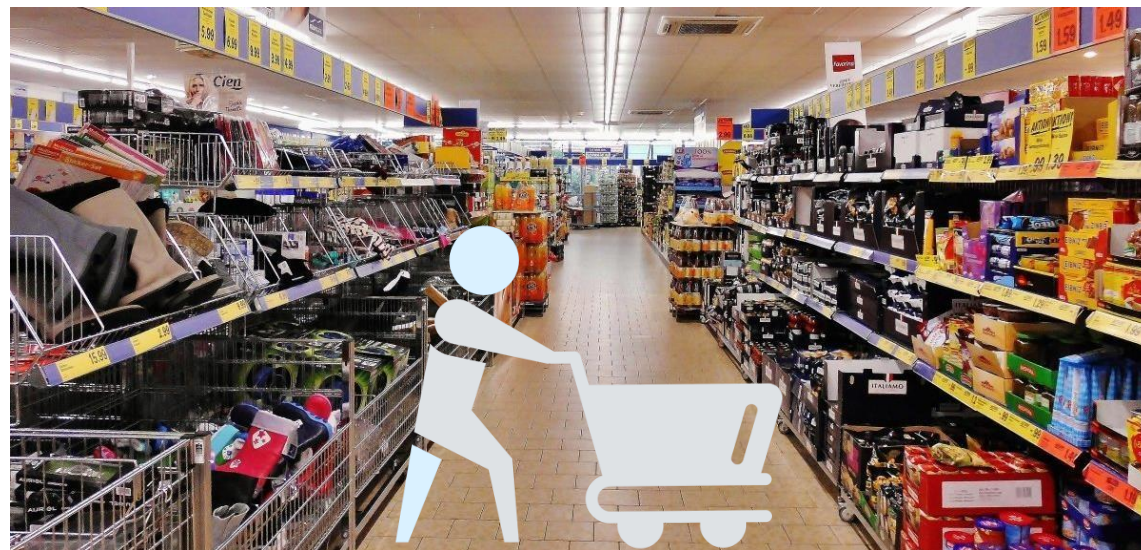
王元宏

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- 5.4 可靠性设计
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- 5.4 可靠性设计
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4



问题：如何带走总价最多的商品？

0-1背包问题形式化定义

- 输入：
 - n 个商品组成的集合 O ，每个商品有两个属性 v_i 和 p_i ，分别表示体积和价格
 - 背包容量 C
- 输出：
 - 求解一个商品子集 $S \subseteq O$ ，令

$$\begin{aligned} \max \quad & \sum_{i \in S} p_i \\ \text{s. t.} \quad & \sum_{i \in S} v_i \leq C \end{aligned}$$

优化目标

约束条件

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

	商品列表	总价格	总体积	说明
方案1	 	34	15	

- 超市允许顾客使用一个体积大小为**13**的背包，选择一件或多件商品带走

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

	商品列表	总价格	总体积	说明
方案1	 	34	15	

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积		商品列表	总价格	总体积	说明
 啤酒	24	10	方案1	 	34	15	错误方案
 汽水	2	3					
 饼干	9	4					
 面包	10	5					
 牛奶	9	4					

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积		商品列表	总价格	总体积	说明
 啤酒	24	10	方案1	 	34	15	错误方案
 汽水	2	3	方案2	 	26	13	
 饼干	9	4					
 面包	10	5					
 牛奶	9	4					

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

	商品列表	总价格	总体积	说明
方案1	 	34	15	错误方案
方案2	 	26	13	可行方案

- 超市允许顾客使用一个体积大小为13的背包，选择一件或多件商品带走

商品	价格	体积		商品列表	总价格	总体积	说明
 啤酒	24	10		 	34	15	错误方案
 汽水	2	3		 	26	13	可行方案
 饼干	9	4		  	28	13	
 面包	10	5					
 牛奶	9	4					

- 超市允许顾客使用一个体积大小为**13**的背包，选择一件或多件商品带走

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

	商品列表	总价格	总体积	说明
方案1	 	34	15	错误方案
方案2	 	26	13	可行方案
方案3	  	28	13	最优方案

问题：如何求解该问题？

- 核心思想：将商品排序，依次挑选
 - 策略1：按商品价格由高到低排序，优先挑选**价格高**的商品

商品	价格	体积
 啤酒	24	10
 面包	10	5
 饼干	9	4
 牛奶	9	4
 汽水	2	3

	商品列表	总体积	总价格	说明



剩余体积：13
商品价值：0

- 核心思想：将商品排序，依次挑选
 - 策略1：按商品价格由高到低排序，优先挑选**价格高**的商品

商品	价格	体积		商品列表	总体积	总价格	说明
 啤酒	24	10		策略1 	10	24	
 面包	10	5	×				
 饼干	9	4	×				
 牛奶	9	4	×				
 汽水	2	3					



剩余体积：3
商品价值：24

- 核心思想：将商品排序，依次挑选
 - 策略1：按商品价格由高到低排序，优先挑选**价格高**的商品

商品	价格	体积		商品列表	总体积	总价格	说明
 啤酒	24	10		策略1  	13	26	
 面包	10	5	×				
 饼干	9	4	×				
 牛奶	9	4	×				
 汽水	2	3					



剩余体积：0
商品价值：26

- 核心思想：将商品排序，依次挑选
 - 策略1：按商品价格由高到低排序，优先挑选**价格高**的商品

商品	价格	体积
 啤酒	24	10
 面包	10	5
 饼干	9	4
 牛奶	9	4
 汽水	2	3

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解



剩余体积：0
商品价值：26

- 核心思想：将商品排序，依次挑选
 - 策略2：按商品体积由小到大排序，优先挑选**体积小**的商品

商品	价格	体积
 汽水	2	3
 饼干	9	4
 牛奶	9	4
 面包	10	5
 啤酒	24	10

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2				



剩余体积：13
商品价值：0

- 核心思想：将商品排序，依次挑选
 - 策略2：按商品体积由小到大排序，优先挑选**体积小**的商品

商品	价格	体积
 汽水	2	3
 饼干	9	4
 牛奶	9	4
 面包	10	5
 啤酒	24	10

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2		3	2	



剩余体积：10
商品价值：2

- 核心思想：将商品排序，依次挑选
 - 策略2：按商品体积由小到大排序，优先挑选**体积小**的商品

商品	价格	体积
 汽水	2	3
 饼干	9	4
 牛奶	9	4
 面包	10	5
 啤酒	24	10

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2	 	7	11	



剩余体积：6
商品价值：11

- 核心思想：将商品排序，依次挑选
 - 策略2：按商品体积由小到大排序，优先挑选**体积小**的商品

商品	价格	体积
 汽水	2	3
 饼干	9	4
 牛奶	9	4
 面包	10	5
 啤酒	24	10

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2	  	11	20	



剩余体积：2
商品价值：20

- 核心思想：将商品排序，依次挑选
 - 策略2：按商品体积由小到大排序，优先挑选**体积小**的商品

商品	价格	体积
 汽水	2	3
 饼干	9	4
 牛奶	9	4
 面包	10	5
 啤酒	24	10

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2	  	11	20	非最优解



剩余体积：2
商品价值：20

- 核心思想：将商品排序，依次挑选
 - 策略3：按商品价值与体积的比由高到低排序，优先挑选**比值高**的商品

商品	价格	体积	比值
 啤酒	24	10	2.4
 饼干	9	4	2.25
 牛奶	9	4	2.25
 面包	10	5	2
 汽水	2	3	0.67

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2	  	11	20	非最优解
策略3				



剩余体积：13
商品价值：0

- 核心思想：将商品排序，依次挑选
 - 策略3：按商品价值与体积的比由高到低排序，优先挑选**比值高**的商品

商品	价格	体积	比值		商品列表	总体积	总价格	说明
 啤酒	24	10	2.4		策略1  	13	26	非最优解
 饼干	9	4	2.25	✗	策略2   	11	20	非最优解
 牛奶	9	4	2.25	✗	策略3 	10	24	
 面包	10	5	2	✗				
 汽水	2	3	0.67					



剩余体积：3
商品价值：24

- 核心思想：将商品排序，依次挑选
 - 策略3：按商品价值与体积的比由高到低排序，优先挑选**比值高**的商品

商品	价格	体积	比值		商品列表	总体积	总价格	说明
 啤酒	24	10	2.4		策略1  	13	26	非最优解
 饼干	9	4	2.25	✗	策略2   	11	20	非最优解
 牛奶	9	4	2.25	✗	策略3  	13	26	
 面包	10	5	2	✗				
 汽水	2	3	0.67					



剩余体积：0
商品价值：26

- 核心思想：将商品排序，依次挑选
 - 策略3：按商品价值与体积的比由高到低排序，优先挑选**比值高**的商品

商品	价格	体积	比值
 啤酒	24	10	2.4
 饼干	9	4	2.25
 牛奶	9	4	2.25
 面包	10	5	2
 汽水	2	3	0.67

	商品列表	总体积	总价格	说明
策略1	 	13	26	非最优解
策略2	  	11	20	非最优解
策略3	 	13	26	非最优解



剩余体积：0
商品价值：26

问题：如何**保证**获得最优解？

- 枚举所有商品组合



背包体积13

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

蛮力枚举



} C_5^1



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

蛮力枚举




背包体积13

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

蛮力枚举




背包体积13

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

蛮力枚举



背包体积13

商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

蛮力枚举



商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

蛮力枚举

- 枚举所有商品组合（共 $2^n - 1$ 种情况），检查体积约束

The diagram illustrates the brute-force enumeration of all possible combinations of 5 items (beer, soda, cookies, bread, milk) in shopping carts. The items are represented by icons: beer (glass), soda (bottle), cookies (round), bread (loaf), and milk (can). The combinations are grouped by the number of items in the cart, labeled as C_5^1 , C_5^2 , C_5^3 , C_5^4 , and C_5^5 . A backpack icon with the label "背包体积13" (Backpack volume 13) is shown next to the groups.

商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

• 枚举所有商品组合（共 $2^n - 1$ 种情况），检查体积约束

10

3

5

4

4

13

15

14

14

8

7

7

9

9

8

17

19

18

19

17

18

12

11

11

13

16

23

21

22

22

26

绿色数字：商品组合体积

背包体积13

商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

蛮力枚举

- 枚举所有商品组合（共 $2^n - 1$ 种情况），检查体积约束



枚举所有商品组合（共 $2^n - 1$ 种情况），检查体积约束





¥24





¥2





¥10





¥9





¥9





¥26

















¥12





¥11





¥11





¥19





¥19





¥18





























¥21





¥20





¥21





¥28



































背包体积13

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

- ### 背包体积13

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

蛮力枚举：递归求解

- 递归函数： $\text{KnapsackSR}(h, i, c)$
 - 在第 h 个到第 i 个商品中，容量为 c 时最优解

$\text{KnapsackSR}(1, 5, 13)$



在第1到5个商品中选择
剩余体积:13

序号	商品	价格	体积
1	 饼干	9	4
2	 面包	10	5
3	 牛奶	9	4
4	 汽水	2	3
5	 啤酒	24	10

蛮力枚举：递归求解

● 递归函数： $\text{KnapsackSR}(h, i, c)$

- 在第 h 个到第 i 个商品中，容量为 c 时最优解
- 选择啤酒： $\text{KnapsackSR}(1, 4, 3) + 24$

$\text{KnapsackSR}(1, 5, 13)$



选择啤酒

$\text{KnapsackSR}(1, 4, 3) + 24$



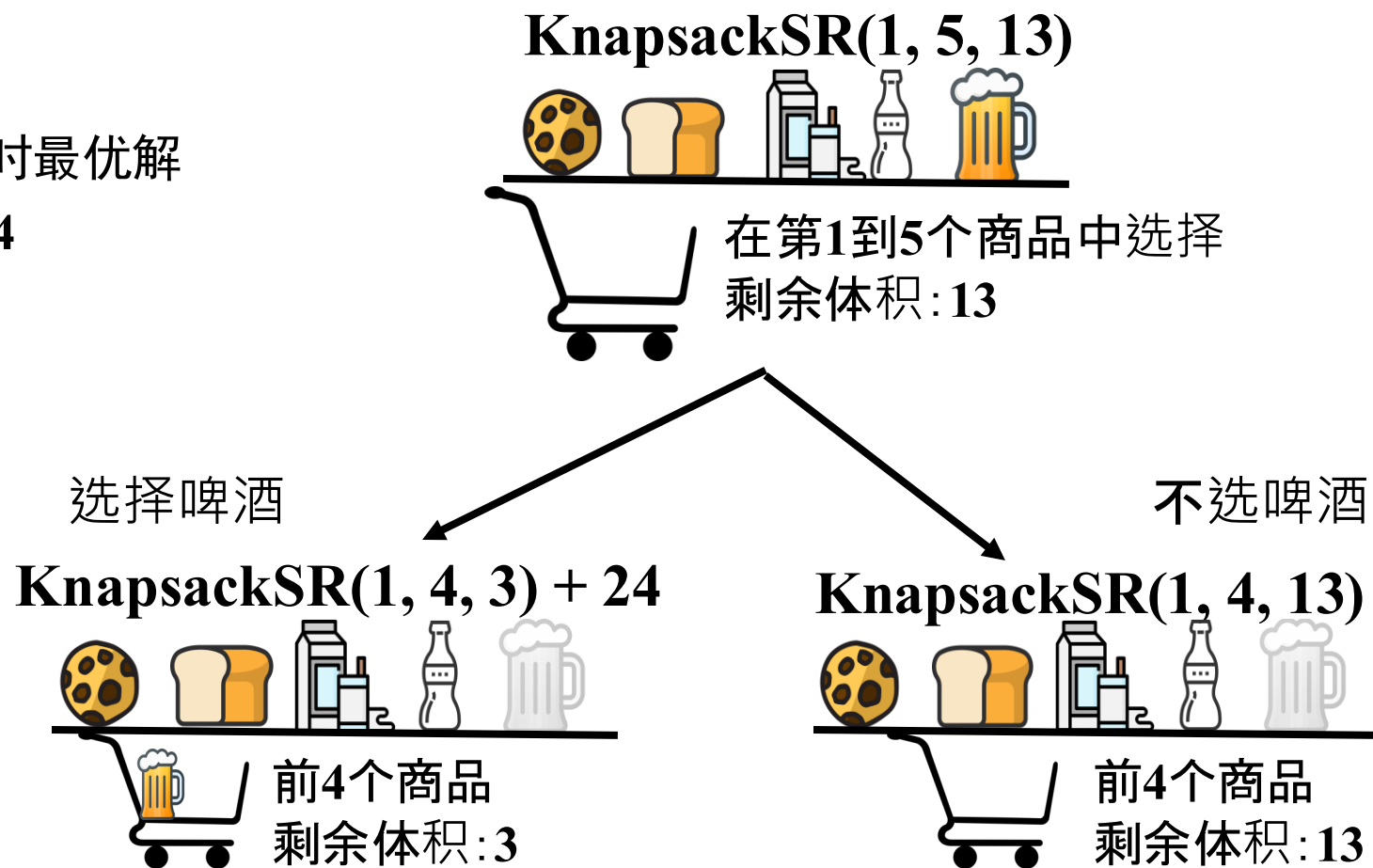
序号	商品	价格	体积
1	饼干	9	4
2	面包	10	5
3	牛奶	9	4
4	汽水	2	3
5	啤酒	24	10

蛮力枚举：递归求解

● 递归函数： $\text{KnapsackSR}(h, i, c)$

- 在第 h 个到第 i 个商品中，容量为 c 时最优解
- **选择**啤酒： $\text{KnapsackSR}(1, 4, 3) + 24$
- **不选择**啤酒： $\text{KnapsackSR}(1, 4, 13)$

序号	商品	价格	体积
1	 饼干	9	4
2	 面包	10	5
3	 牛奶	9	4
4	 汽水	2	3
5	 啤酒	24	10

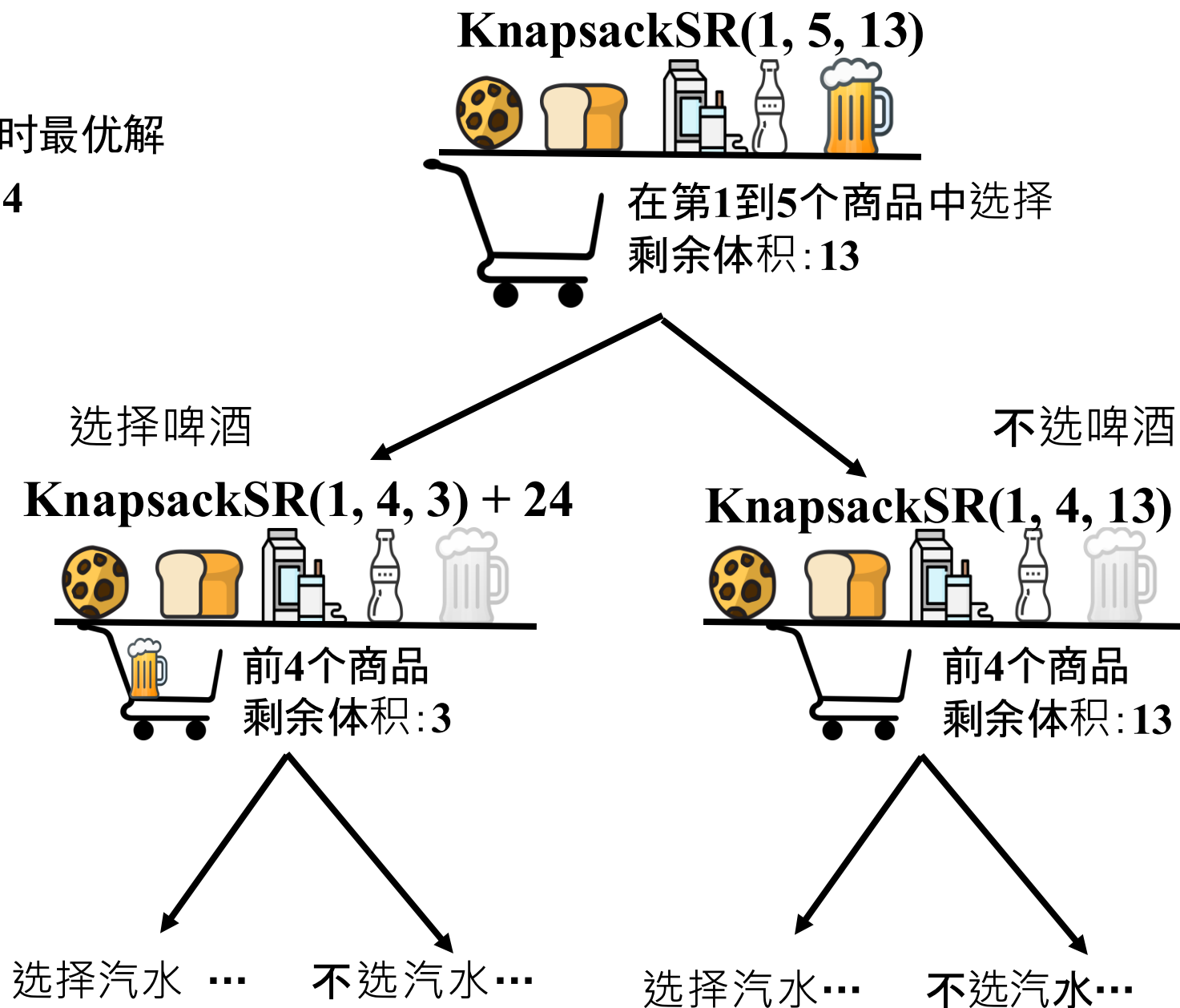


蛮力枚举：递归求解

● 递归函数： $\text{KnapsackSR}(h, i, c)$

- 在第 h 个到第 i 个商品中，容量为 c 时最优解
- **选择**啤酒： $\text{KnapsackSR}(1, 4, 3) + 24$
- **不选择**啤酒： $\text{KnapsackSR}(1, 4, 13)$

序号	商品	价格	体积
1	 饼干	9	4
2	 面包	10	5
3	 牛奶	9	4
4	 汽水	2	3
5	 啤酒	24	10

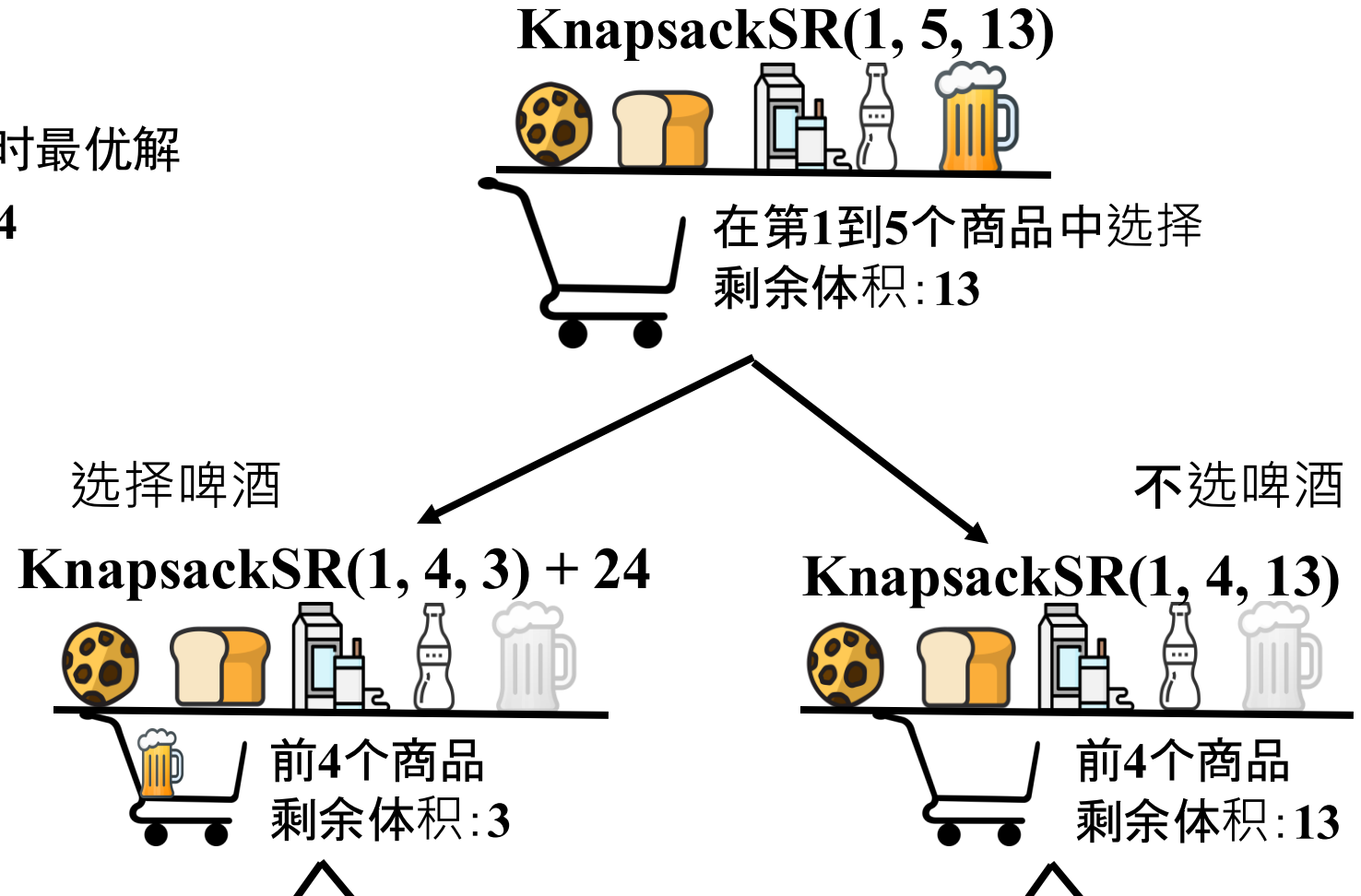


蛮力枚举：递归求解

● 递归函数： $\text{KnapsackSR}(h, i, c)$

- 在第 h 个到第 i 个商品中，容量为 c 时最优解
- **选择**啤酒： $\text{KnapsackSR}(1, 4, 3) + 24$
- **不选择**啤酒： $\text{KnapsackSR}(1, 4, 13)$

序号	商品	价格	体积
1	 饼干	9	4
2	 面包	10	5
	 牛奶		
	 汽水		
	 啤酒		



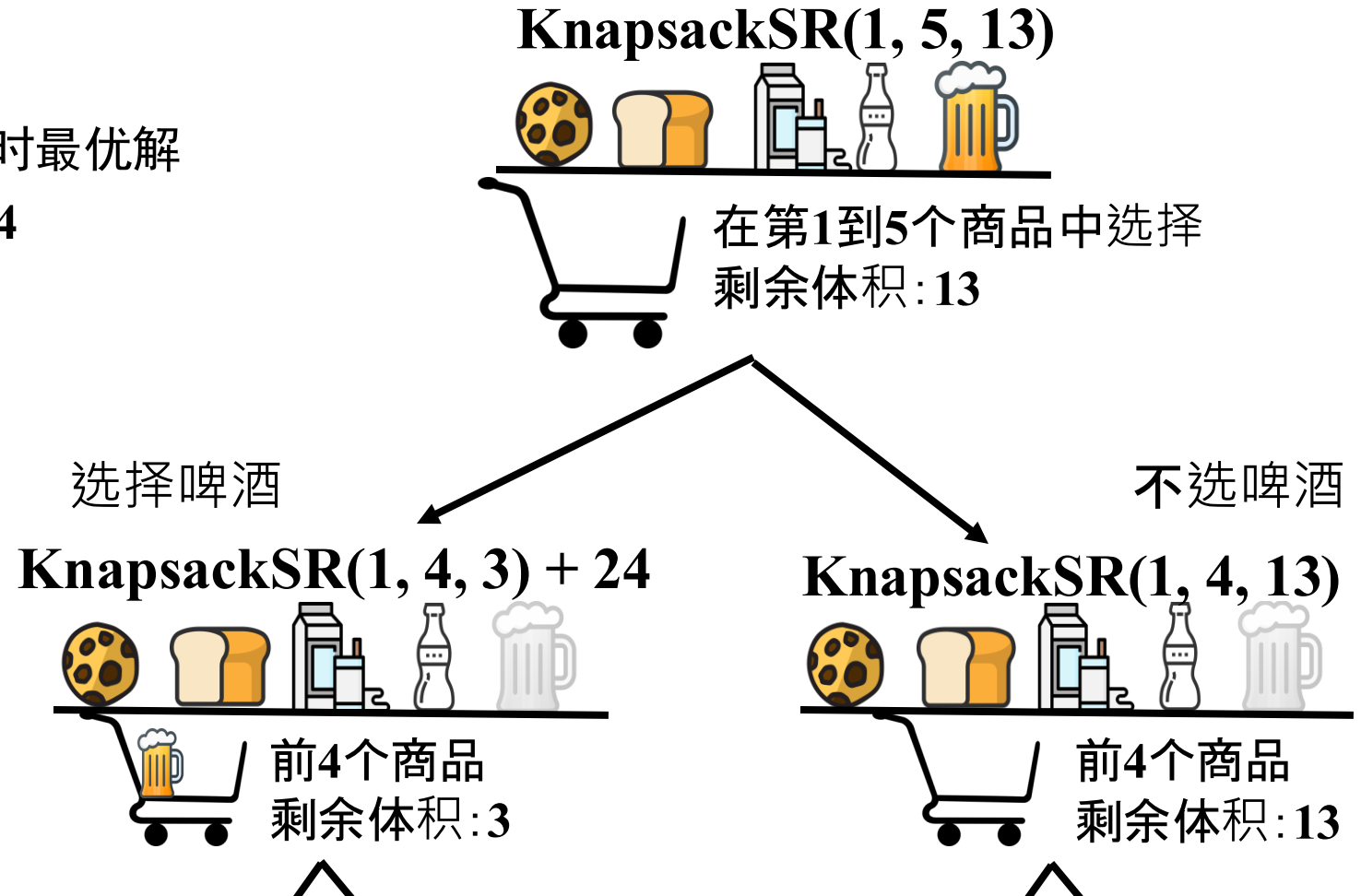
$$\text{KnapsackSR}(1, 5, 13) = \max\{\text{KnapsackSR}(1, 4, 3) + 24, \text{KnapsackSR}(1, 4, 13)\}$$

蛮力枚举：递归求解

● 递归函数： $\text{KnapsackSR}(h, i, c)$

- 在第 h 个到第 i 个商品中，容量为 c 时最优解
- **选择**啤酒： $\text{KnapsackSR}(1, 4, 3) + 24$
- **不选择**啤酒： $\text{KnapsackSR}(1, 4, 13)$

序号	商品	价格	体积
1	 饼干	9	4
2	 面包	10	5
	 牛奶		
	 汽水		
	 啤酒		



$$\text{KnapsackSR}(h, i, c) = \max\{\text{KnapsackSR}(h, i - 1, c - v_i) + 24, \text{KnapsackSR}(h, i - 1, c)\}$$

- $\text{KnapsackSR}(h, i, c)$: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 商品集合 $\{h, \dots, i\}$, 背包容量 c

输出: 最大总价格 P

```
if  $c < 0$  then  
  | return  $-\infty$   
end
```

超出背包容量限制

```
if  $i \leq h - 1$  then  
  | return 0  
end
```

end

$P_1 \leftarrow \text{KnapsackSR}(h, i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(h, i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

- $\text{KnapsackSR}(h, i, c)$: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 商品集合 $\{h, \dots, i\}$, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then
| return $-\infty$

end

if $i \leq h - 1$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(h, i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(h, i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

所有商品已决策完成

蛮力枚举：伪代码

- $\text{KnapsackSR}(h, i, c)$: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 商品集合 $\{h, \dots, i\}$, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then
| return $-\infty$

end

if $i \leq h - 1$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(h, i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(h, i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

选则商品 i

- **KnapsackSR(h, i, c)**: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 商品集合 $\{h, \dots, i\}$, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then
| return $-\infty$

end

if $i \leq h - 1$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(h, i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(h, i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

不选商品 i

- **KnapsackSR(h, i, c)**: 在第 h 个到第 i 个商品中，容量为 c 时最优解

输入: 商品集合 $\{h, \dots, i\}$, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then
| return $-\infty$

end

if $i \leq h - 1$ then

| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(h, i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(h, i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

简化描述

蛮力枚举：伪代码

- $\text{KnapsackSR}(h, i, c)$: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 前 i 个商品, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then

| return $-\infty$

end

if $i \leq 0$ then

| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

简化描述

- $\text{KnapsackSR}(h, i, c)$: 在第 h 个到第 i 个商品中, 容量为 c 时最优解

输入: 前 i 个商品, 背包容量 c

输出: 最大总价格 P

if $c < 0$ then
| return $-\infty$

end

if $i \leq 0$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackSR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackSR}(i - 1, c)$

$P \leftarrow \max\{P_1 + p_i, P_2\}$

return P

确定最优子问题

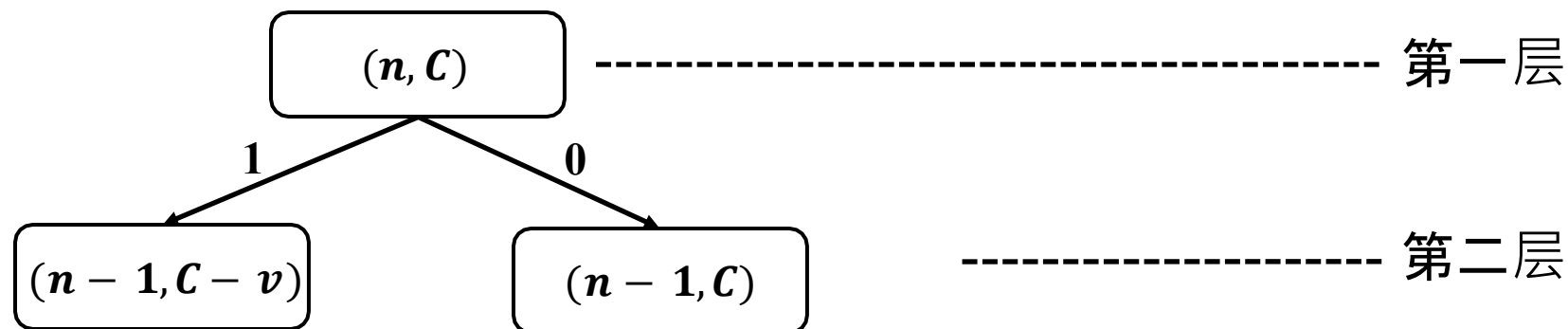
- 递归树
 - 例：商品体积均为 v

(n, C)

----- 第一层

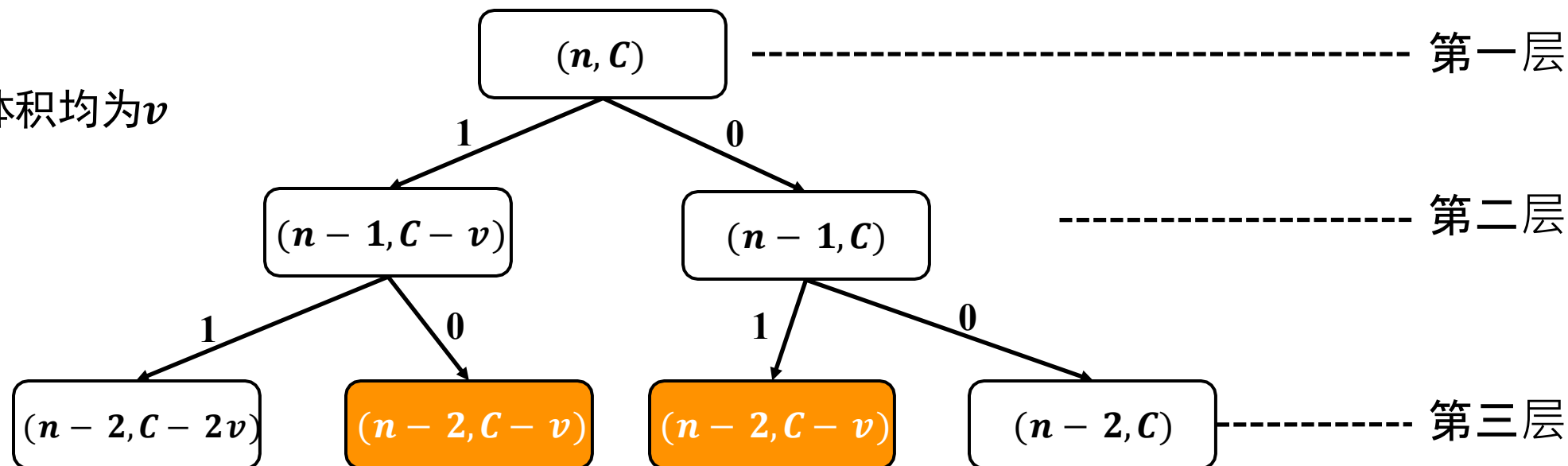
- 递归树

- 例：商品体积均为 v



- 递归树

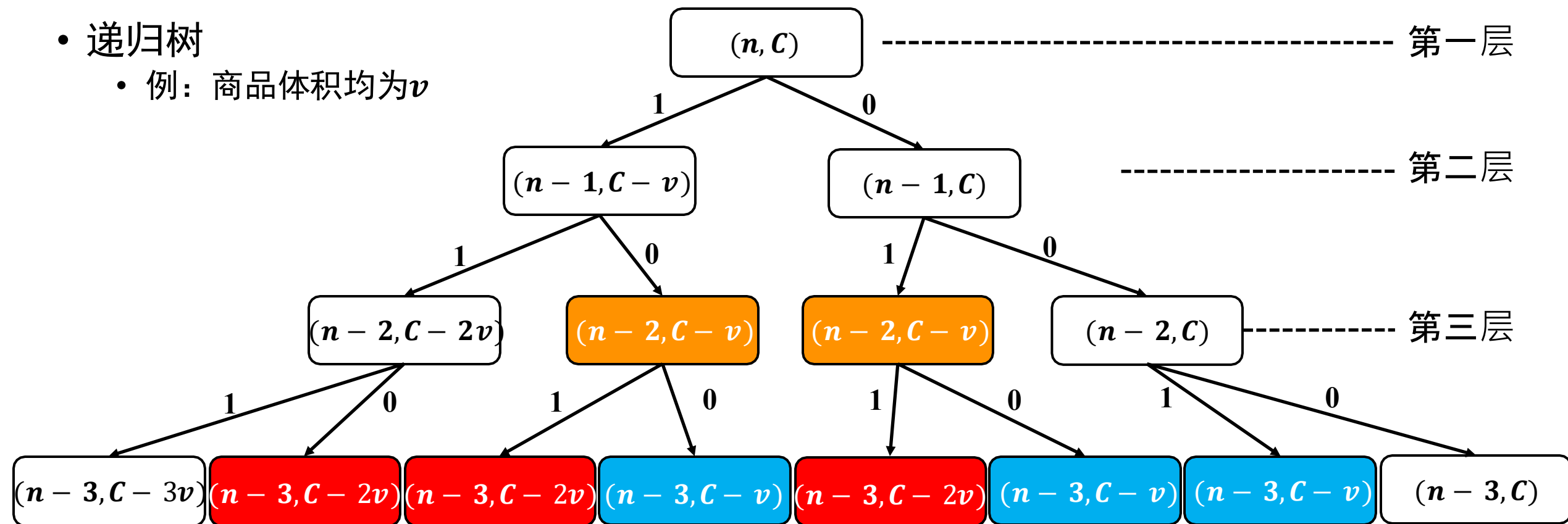
- 例：商品体积均为 v



蛮力枚举：复杂度

- 递归树

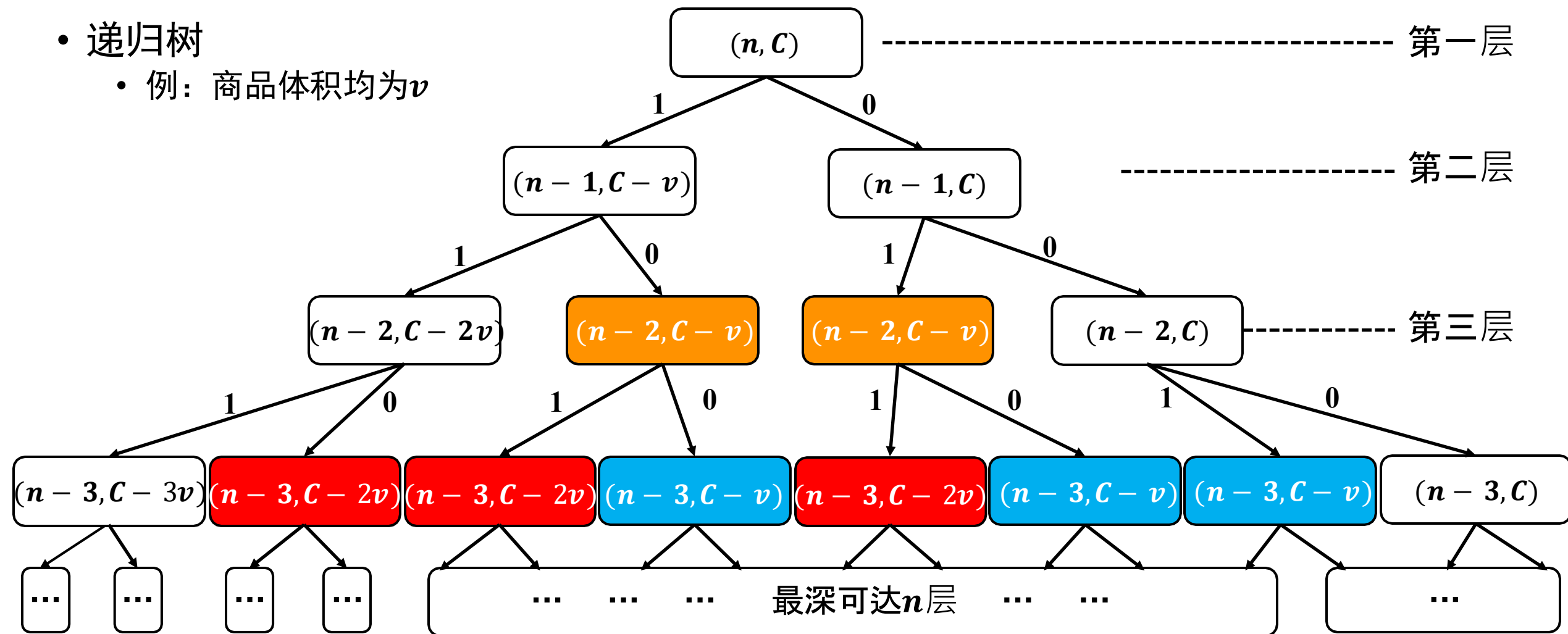
- 例：商品体积均为 v



蛮力枚举：复杂度

- 递归树

- 例：商品体积均为 v

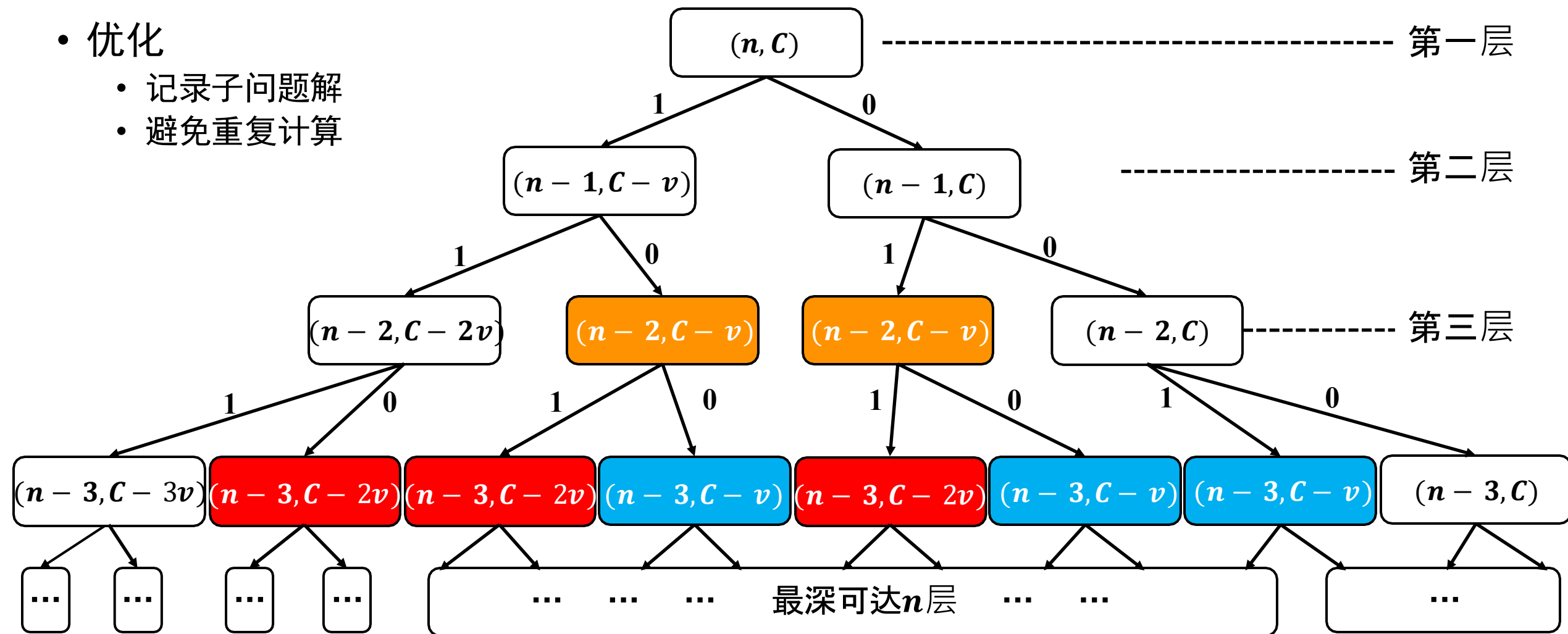


● 重复求解大量子问题 $O(2^n)$

问题：如何优化？

- 优化

- 记录子问题解
- 避免重复计算



输入: 商品集合 $\{1, \dots, i\}$, 背包容量 c

输出: 最大总价格 $P[i, c]$

if $c < 0$ then
| return $-\infty$

end

if $i \leq 0$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackMR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackMR}(i - 1, c)$

$P[i, c] \leftarrow \max\{P_1 + p_i, P_2\}$

return $P[i, c]$

输入: 商品集合 $\{1, \dots, i\}$, 背包容量 c

输出: 最大总价格 $P[i, c]$

if $c < 0$ then
| return $-\infty$

end

if $i \leq 0$ then
| return 0

end

$P_1 \leftarrow \text{KnapsackMR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackMR}(i - 1, c)$

$P[i, c] \leftarrow \max\{P_1 + p_i, P_2\}$

return $P[i, c]$

构造备忘录 $P[i, c]$

$P[i, c]$ 表示在前 i 个商品中选择,
背包容量为 c 时的最优解

输入: 商品集合 $\{1, \dots, i\}$, 背包容量 c

输出: 最大总价格 $P[i, c]$

if $c < 0$ then
| return $-\infty$

end

if $i \leq 0$ then
| return 0

end

if $P[i, c] \neq \text{NULL}$ then
| return $P[i, c]$

end

$P_1 \leftarrow \text{KnapsackMR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackMR}(i - 1, c)$

$P[i, c] \leftarrow \max\{P_1 + p_i, P_2\}$

return $P[i, c]$

构造备忘录 $P[i, c]$

$P[i, c]$ 表示在前 i 个商品中选择,
背包容量为 c 时的最优解

带备忘递归：伪代码

输入: 商品集合 $\{1, \dots, i\}$, 背包容量 c

输出: 最大总价格 $P[i, c]$

if $c < 0$ then
| return $-\infty$

end

if $i \leq 0$ then
| return 0

end

if $P[i, c] \neq \text{NULL}$ then
| return $P[i, c]$

end

$P_1 \leftarrow \text{KnapsackMR}(i - 1, c - v_i)$

$P_2 \leftarrow \text{KnapsackMR}(i - 1, c)$

$P[i, c] \leftarrow \max\{P_1 + p_i, P_2\}$

return $P[i, c]$

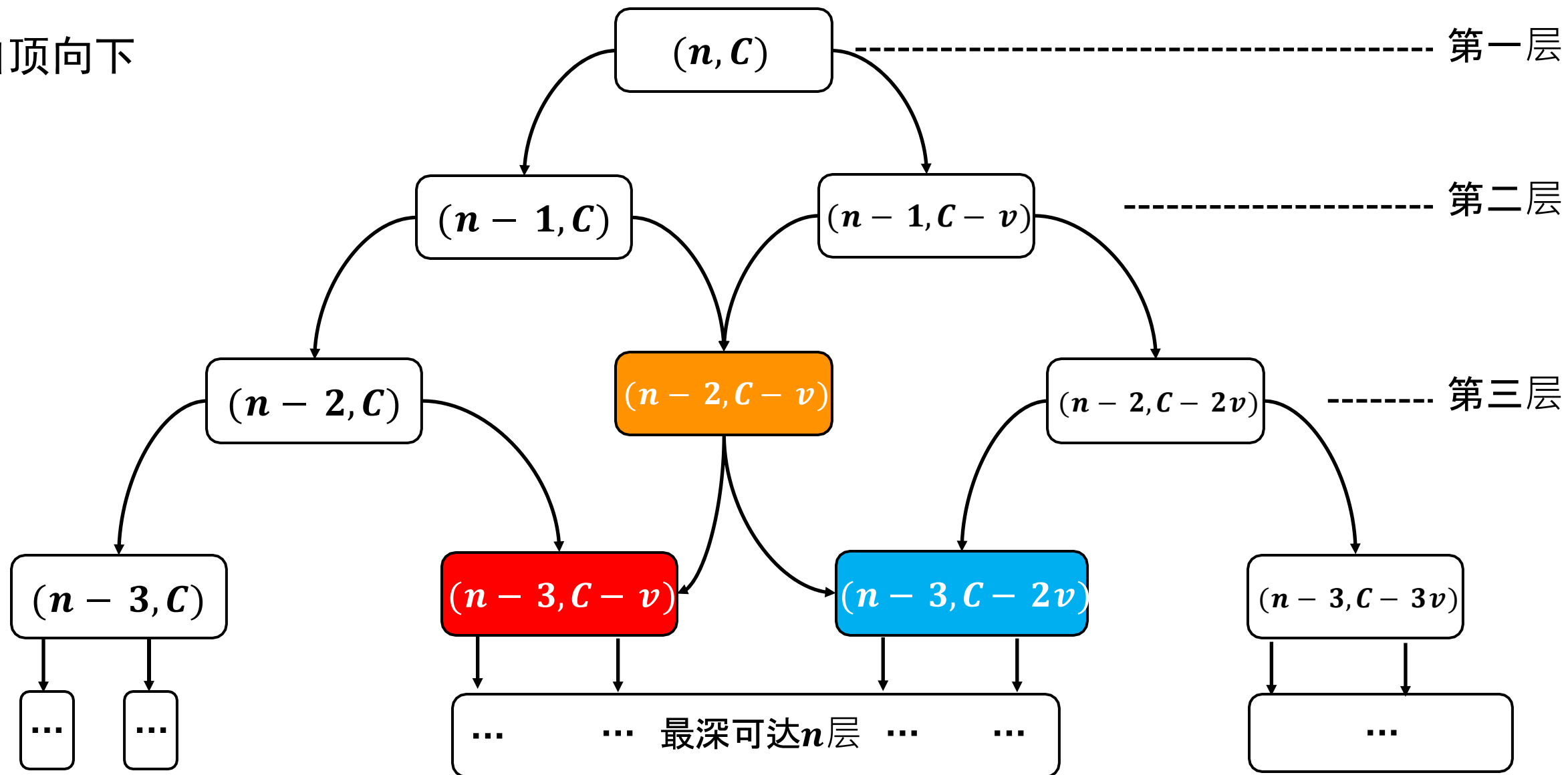
重复子问题

构造备忘录 $P[i, c]$

$P[i, c]$ 表示在前 i 个商品中选择,
背包容量为 c 时的最优解

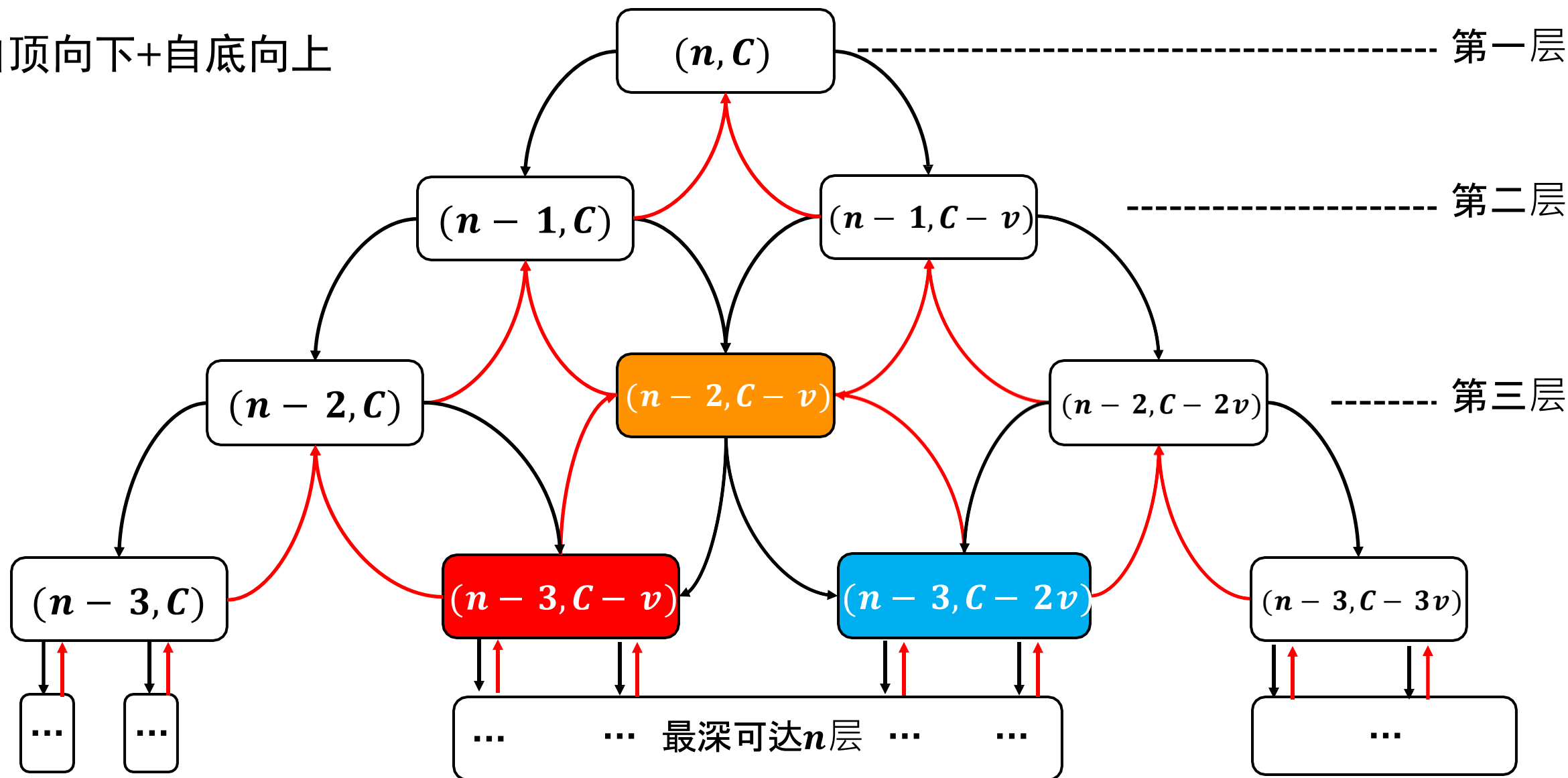
带备忘递归：计算顺序

- 自顶向下



带备忘递归：计算顺序

- 自顶向下+自底向上



问题：是否可以不递归（为什么？），直接求解 $P[i, c]$ ？

● 初始化

- 容量为0时: $P[i, 0] = 0$
- 没有商品时: $P[0, c] = 0$

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

- 确定计算顺序

- $P[i-1, c-v_i]$ 和 $P[i-1, c]$ 位于 $P[i, c]$ 的左上方

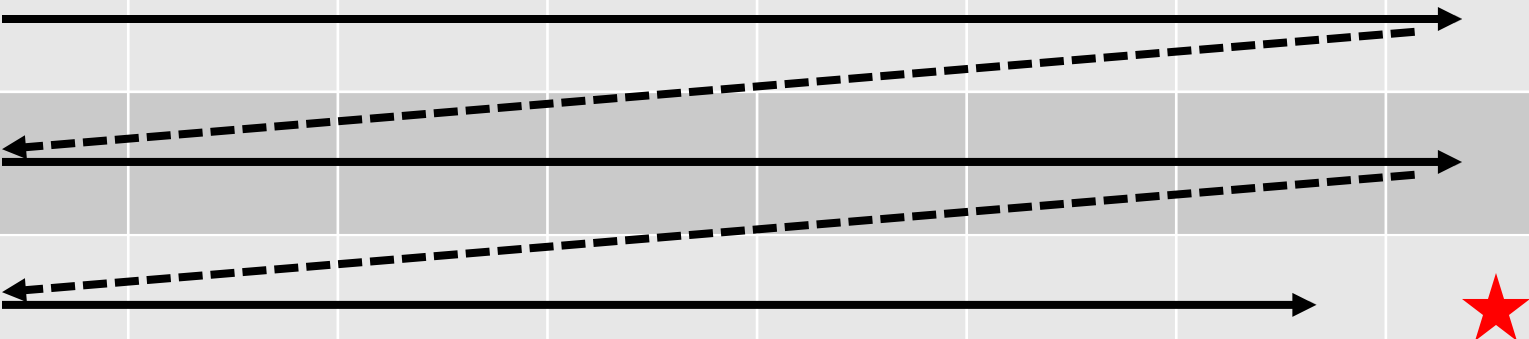
$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

问题：观察子问题依赖关系，如何确定计算顺序？

- 确定计算顺序

- 按从左到右、从上到下的顺序计算

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								



问题：观察子问题依赖关系，如何确定计算顺序？

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$$C = 13$$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

初始化

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

$v[i] > c$

递推公式： $P[i, c] = \max\{P[i-1, c-v[i]] + p[i], P[i-1, c]\}$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

$v[i] > c$

递推公式： $P[i, c] = \max\{P[i - 1, c - v[i]] + p[i], P[i - 1, c]\}$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{P[i - 1, c - v[i]] + p[i], P[i - 1, c]\}$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{P[0, 0] + p[1], P[0, 10]\}$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24			
2	0													
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{0 + 24, 0\}$

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0											
3	0													
4	0													
5	0													

$v[i] > c$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2										
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{0 + 2, 0\}$

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24			
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{0 + 2, 24\}$

$C = 13$

[illegible]

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0													
4	0													
5	0													

递推公式： $P[i, c] = \max\{24 + 2, 24\}$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2										
4	0													
5	0													

$v[i] > c$

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9									
4	0													
5	0													

递推公式： $P[i, c] = \max\{0 + 9, 2\}$

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9									
5	0													

$v[i] > c$

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2										

$v[i] > c$

$C = 13$

[illegible]

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10								

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10							

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11						

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18					

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19				

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24			

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24		

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$$C = 13$$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

最优解

问题：如何确定选取了哪些商品？

- 递推公式

- $P[i, c] = \max\{P[i - 1, c - v_i] + p_i, P[i - 1, c]\}$

- 决策过程

- $Rec[i, c] = \begin{cases} 1, & \text{选择商品} \\ 0, & \text{不选商品} \end{cases}$

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

$Rec[i, c] = 1$ (diagonal arrow from (1, 3) to (n, 10))

$Rec[i, c] = 0$ (vertical arrow from (0, 10) to (n, 10))

$P[i, c]$ (at (n, 10))

- 递推公式

- $$P[i, c] = \max\{P[i - 1, c - v_i] + p_i, P[i - 1, c]\}$$

- 决策过程

- $$Rec[i, c] = \begin{cases} 1, & \text{选择商品} \\ 0, & \text{不选商品} \end{cases}$$

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

- 递推公式

- $P[i, c] = \max\{P[i - 1, c - v_i] + p_i, P[i - 1, c]\}$

- 决策过程

- $Rec[i, c] = \begin{cases} 1, & \text{选择商品} \\ 0, & \text{不选商品} \end{cases}$

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

$Rec[i, c] = 1$ (diagonal arrow pointing to $P[i, c]$)

$Rec[i, c] = 0$ (vertical arrow pointing to $P[i, c]$)

- 递推公式
 - $P[i, c] = \max\{P[i - 1, c - v_i] + p_i, P[i - 1, c]\}$
- 回溯解决方案
 - 倒序判断是否选择商品
 - 根据选择结果，确定最优子问题

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

$Rec[i, c] = 1$ (diagonal arrow from $P[i, c]$ to $P[i-1, c-v_i]$)

$Rec[i, c] = 0$ (vertical arrow from $P[i, c]$ to $P[i-1, c]$)

算法实例

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0				
2	0													
3	0													
4	0													
5	0													

$v[i] > c$

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0				
1	0													
2	0													
3	0													
4	0													
5	0													

当前状态最优解不包含商品 i

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$$C = 13$$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24			
2	0													
3	0													
4	0													
5	0													

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	1			
1	0													
2	0													
3	0													
4	0													
5	0													

当前状态最优解包含商品*i*

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24			
2	0	0	0	2	2	2	2	2	2	2	24			
3	0													
4	0													
5	0													

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
1	0	0	0	1	1	1	1	1	1	1	0			
2	0													
3	0													
4	0													
5	0													

当前状态最优解不包含商品*i*

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	

v_i	10	3	4	5	4
p_i	24	2	9	10	9

$C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

算法实例

v_i	10	3	4	5	4
p_i	24	2	9	10	9

选取的商品：**5** $C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

最优解包含商品5

v_i	10	3	4	5	4
p_i	24	2	9	10	9

选取的商品：5, 4 $C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

最优解包含商品4

v_i	10	3	4	5	4
p_i	24	2	9	10	9

选取的商品：5, 4, 3 $C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	最优解包含商品3					0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

v_i	10	3	4	5	4
p_i	24	2	9	10	9

选取的商品：5, 4, 3 $C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

最优解包含商品2

v_i	10	3	4	5	4
p_i	24	2	9	10	9

选取的商品：5, 4, 3 $C = 13$

$P[i, c]$	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 0$	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	24	24	24	24
2	0	0	0	2	2	2	2	2	2	2	24	24	24	26
3	0	0	0	2	9	9	9	11	11	11	24	24	24	26
4	0	0	0	2	9	10	10	11	12	19	24	24	24	26
5	0	0	0	2	9	10	10	11	18	19	24	24	24	28

Rec	$c = 0$	1	2	3	4	5	6	7	8	9	10	11	12	13
$i = 1$	0	0	0	0	0	0	0	0	0	0	1	1	1	1
2	0	0	0	1	1	1	1	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	1	1	0	0	0	0
4	0	0	0	0	0	1	1	0	1	1	0	0	0	0
5	0	0	0	0	0	0	0	0	1	0	0	0	0	1

- KnapsackDP(n, p, v, C)

输入: 商品数量 n , 各商品的价值 p , 各商品的体积 v , 背包容量 C

输出: 商品价格的最大值, 最优解方案

//初始化

创建二维数组 $P[0..n, 0..C]$ 和 $Rec[0..n, 0..C]$

for $i \leftarrow 0$ to C do

 | $P[0, i] \leftarrow 0$

end

for $i \leftarrow 0$ to n do

 | $P[i, 0] \leftarrow 0$

end

初始化

- KnapsackDP(n, p, v, C)

//求解表格

for $i \leftarrow 1$ to n do

 for $c \leftarrow 1$ to C do

 if $(v[i] \leq c)$ and

$(p[i] + P[i - 1, c - v[i]] > P[i - 1, c])$ then

$P[i, c] \leftarrow p[i] + P[i - 1, c - v[i]]$

$Rec[i, c] \leftarrow 1$

 end

 else

$P[i, c] \leftarrow P[i - 1, c]$

$Rec[i, c] \leftarrow 0$

 end

end

end

依次计算子问题

- KnapsackDP(n, p, v, C)

//求解表格

for $i \leftarrow 1$ to n do

 for $c \leftarrow 1$ to C do

 if $(v[i] \leq c)$ and
 $(p[i] + P[i - 1, c - v[i]] > P[i - 1, c])$ then

选择商品 i

$P[i, c] \leftarrow p[i] + P[i - 1, c - v[i]]$

$Rec[i, c] \leftarrow 1$

 end

 else

$P[i, c] \leftarrow P[i - 1, c]$

$Rec[i, c] \leftarrow 0$

 end

end

end

- **KnapsackDP(n, p, v, C)**

//求解表格

for $i \leftarrow 1$ **to** n **do**

for $c \leftarrow 1$ **to** C **do**

if $(v[i] \leq c)$ **and**

$(p[i] + P[i - 1, c - v[i]] > P[i - 1, c])$ **then**

$P[i, c] \leftarrow p[i] + P[i - 1, c - v[i]]$

$Rec[i, c] \leftarrow 1$

end

else

$P[i, c] \leftarrow P[i - 1, c]$

$Rec[i, c] \leftarrow 0$

end

end

end

记录价格和决策

- KnapsackDP(n, p, v, C)

//求解表格

```
for  $i \leftarrow 1$  to  $n$  do
  for  $c \leftarrow 1$  to  $C$  do
    if  $(v[i] \leq c)$  and
       $(p[i] + P[i - 1, c - v[i]] > P[i - 1, c])$  then
      |  $P[i, c] \leftarrow p[i] + P[i - 1, c - v[i]]$ 
      |  $Rec[i, c] \leftarrow 1$ 
    end
    else
    |  $P[i, c] \leftarrow P[i - 1, c]$ 
    |  $Rec[i, c] \leftarrow 0$ 
    end
  end
end
end
```

不选商品 i

- **KnapsackDP(n, p, v, C)**

//输出最优解方案

$K \leftarrow C$

for $i \leftarrow n$ **to** 1 **do**

if $Rec[i, K] = 1$ **then**

 print **选择商品** i

$K \leftarrow K - v[i]$

end

else

 print **不选商品** i

end

end

return $P[n, C]$

倒序判断是否选择该商品

- KnapsackDP(n, p, v, C)

//输出最优解方案

$K \leftarrow C$

for $i \leftarrow n$ to 1 do

 if $Rec[i, K] = 1$ then

 print 选择商品 i

$K \leftarrow K - v[i]$

 end

 else

 print 不选商品 i

 end

end

return $P[n, C]$

选择商品 i

- KnapsackDP(n, p, v, C)

//输出最优解方案

$K \leftarrow C$

for $i \leftarrow n$ to 1 do

 if $Rec[i, K] = 1$ then

 print 选择商品 i

$K \leftarrow K - v[i]$

 end

 else

 print 不选商品 i

 end

end

return $P[n, C]$

回溯子问题

- KnapsackDP(n, p, v, C)

//输出最优解方案

$K \leftarrow C$

for $i \leftarrow n$ to 1 do

 if $Rec[i, K] = 1$ then

 print 选择商品 i

$K \leftarrow K - v[i]$

 end

 else

 print 不选商品 i

 end

end

return $P[n, C]$

不选商品 i

//求解表格

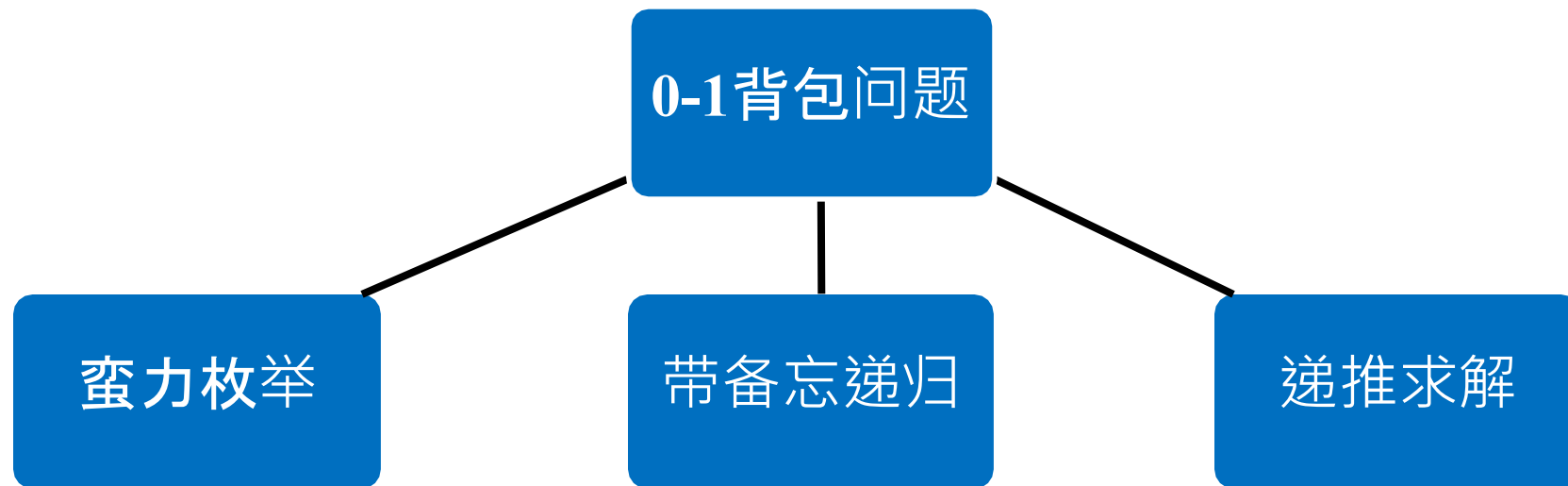
```
for  $i \leftarrow 1$  to  $n$  do
  for  $c \leftarrow 1$  to  $C$  do
    if  $(v[i] \leq c)$  and
       $(p[i] + P[i - 1, c - v[i]] > P[i - 1, c])$  then
      |  $P[i, c] \leftarrow p[i] + P[i - 1, c - v[i]]$ 
      |  $Rec[i, c] \leftarrow 1$ 
    end
    else
      |  $P[i, c] \leftarrow P[i - 1, c]$ 
      |  $Rec[i, c] \leftarrow 0$ 
    end
  end
end
```

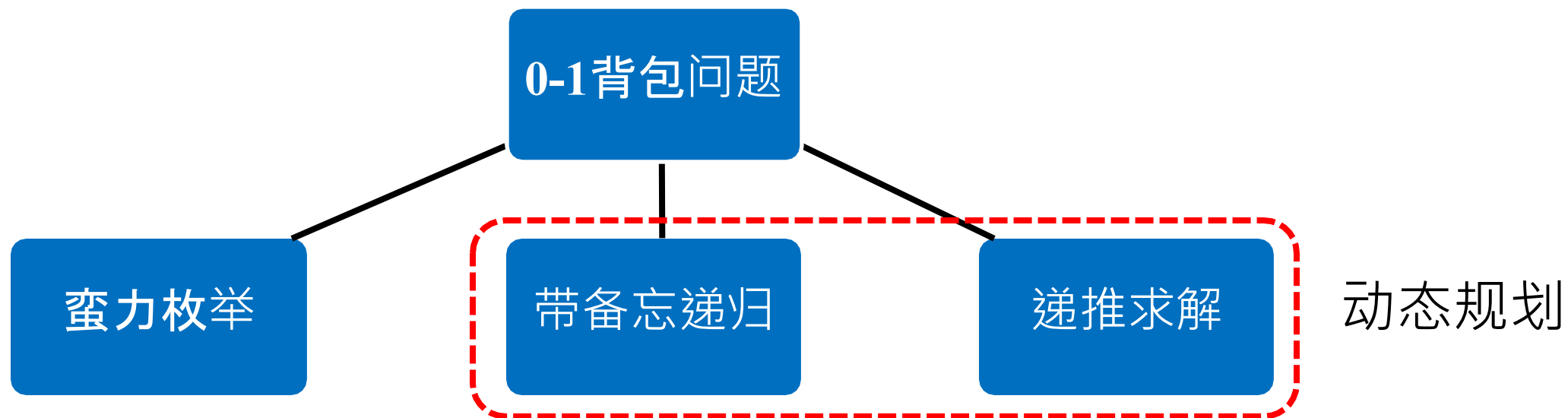
时间复杂度 : $O(n \cdot C)$

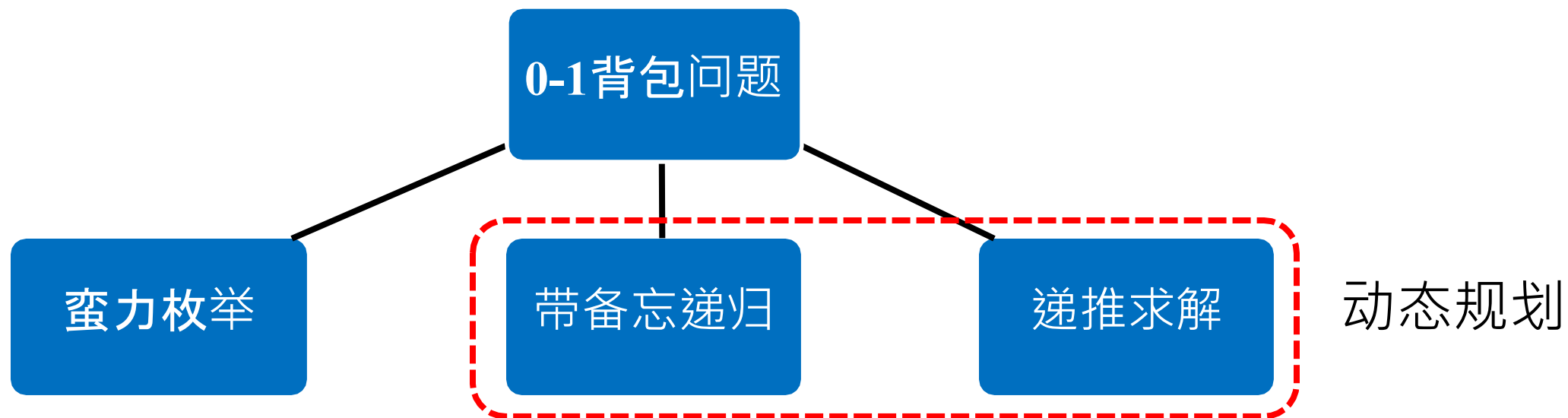
$O(C)$

$O(n \cdot C)$

思考：该算法是多项式时间算法吗？







	带备忘递归	递推求解
相同	分解问题，寻找关系	
不同	自顶向下	自底向上 更高效！

- 5.1 0-1背包问题
- **5.2 动态规划主要思想**
- 5.3 再论背包问题
- 5.4 可靠性设计
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

- 一类多阶段决策问题
 - 它可以被分成若干个阶段，每一个阶段都需要作出决策， i 阶段的决策仅依赖于 i 阶段当前的状态（无后效性）
 - 当每个阶段的决策都确定后，问题得到一个决策序列，进而计算出一个解
- 动态规划方法适用于解决求最优解的多阶段决策问题
 - 寻找最优解和最优决策序列

问题描述中一定存在目标函数

也称为多阶段决策优化问题，
或组合优化问题

- 一类多阶段决策问题
 - 它可以被分成若干个阶段，每一个阶段都需要作出决策， i 阶段的决策仅依赖于 i 阶段当前的状态（无后效性）
 - 当每个阶段的决策都确定后，问题得到一个决策序列，进而计算出一个解
- 动态规划方法适用于解决求最优解的多阶段决策问题
 - 寻找最优解和最优决策序列
- 最优子结构性质（最优性原理）
 - 问题的最优决策序列包含子问题的最优决策序列。可以通过子问题的最优决策序列，推导出问题的最优决策序列

思考：任意子问题的最优决策序列都是原问题的部分最优决策序列么？

- 一类多阶段决策问题
 - 它可以被分成若干个阶段，每一个阶段都需要作出决策， i 阶段的决策仅依赖于 i 阶段当前的状态（无后效性）
 - 当每个阶段的决策都确定后，问题得到一个决策序列，进而计算出一个解
- 动态规划方法适用于解决求最优解的多阶段决策问题
 - 寻找最优解和最优决策序列
- 最优子结构性质（最优性原理）
 - 问题的最优解包含子问题的最优解。可以通过子问题的最优解，推导出问题的最优解
- 重复子问题（不一定满足）
 - 不同的决策序列，到达某个相同的阶段时，可能会产生重复的状态

20世纪50年代，贝尔曼等人根据多阶段决策问题的特性，提出了解决这类问题的“最优性原理”，从而创建了一种新的算法设计方法——**动态规划**

- 动态规划方法求解多阶段决策问题时：
 - 证明多阶段决策(最优)问题满足最优性原理
 - 设计递推关系式
 - 自底向上计算
 - 最优方案追踪



理查德·贝尔曼
1920年—1984年

- 给出问题表示

- $P[i, c]$: 前 i 个商品可选、背包容量为 c 时的最大总价格

- 明确原始问题

- $P[n, C]$: 前 n 个商品可选、背包容量为 C 时的最大总价格

问题结构分析



递推关系建立



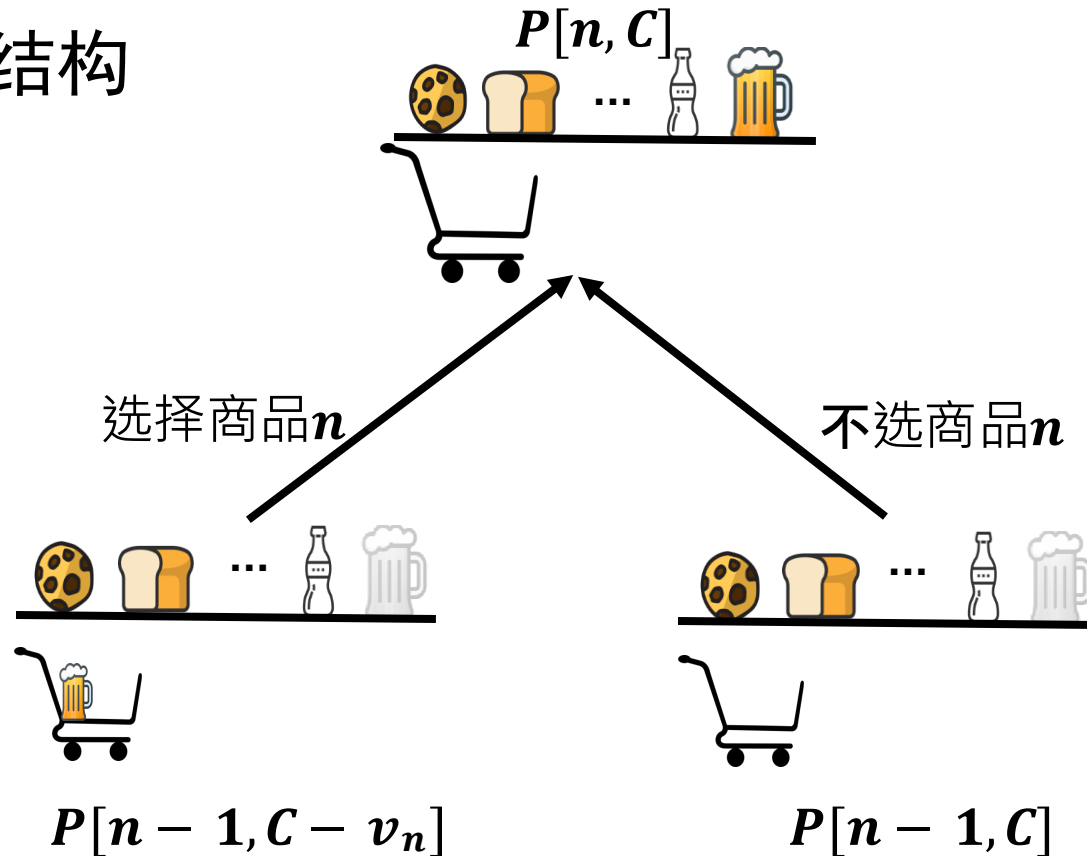
自底向上计算



最优方案追踪

动态规划：一般步骤

- 分析最优（子）结构



问题结构分析

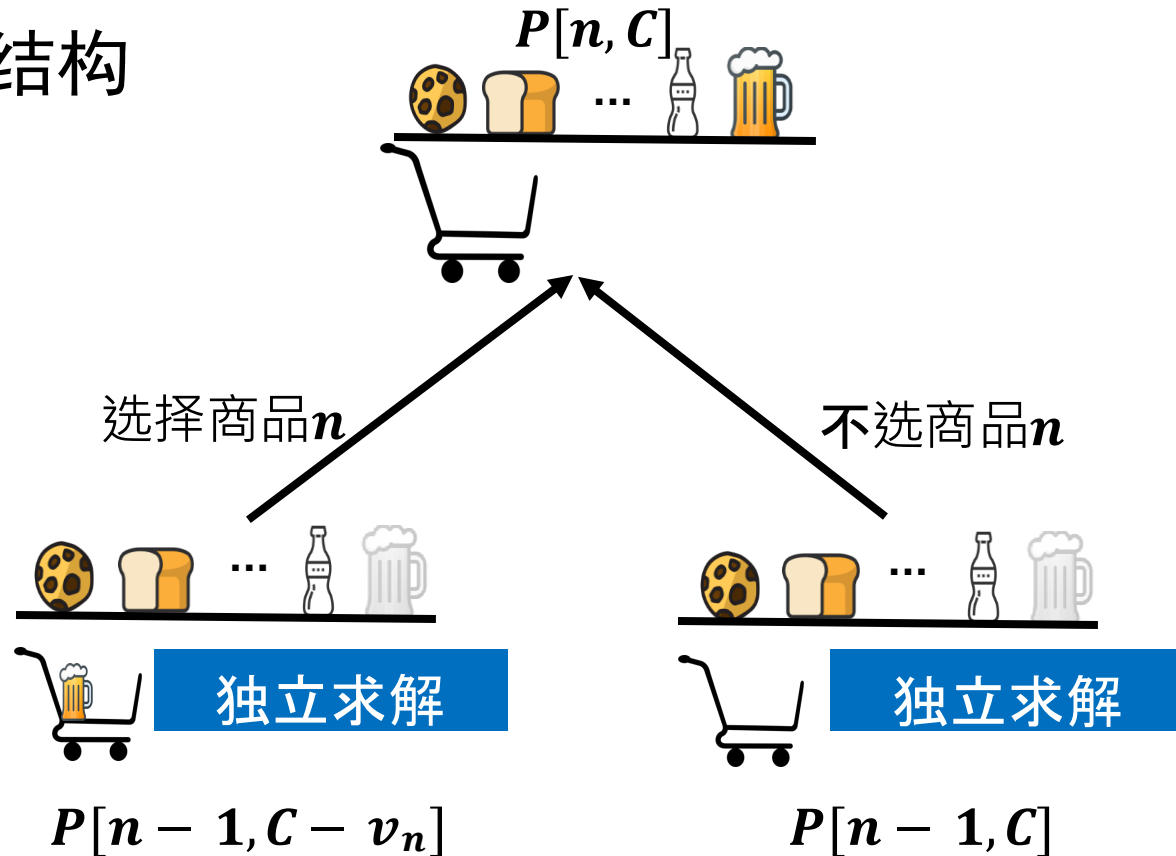
递推关系建立

自底向上计算

最优方案追踪

动态规划：一般步骤

- 分析最优（子）结构



问题结构分析

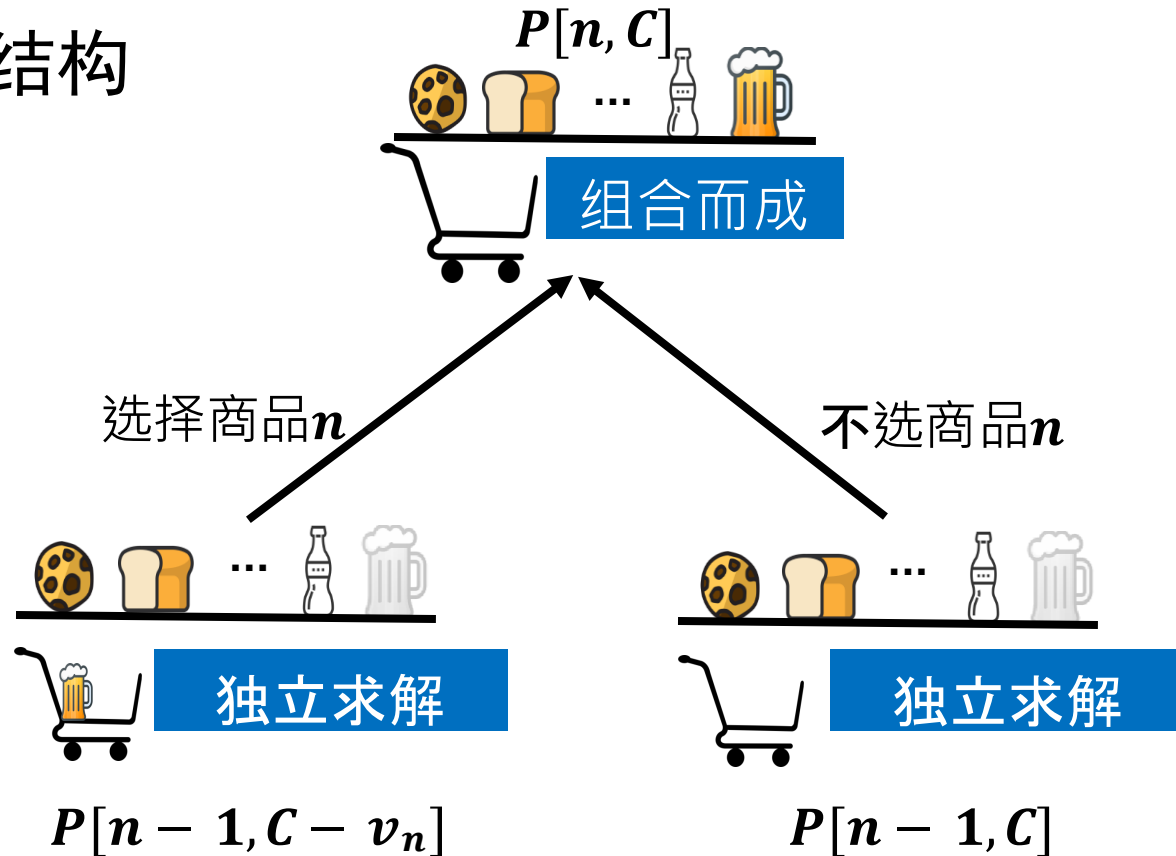
递推关系建立

自底向上计算

最优方案追踪

动态规划：一般步骤

- 分析最优（子）结构



满足无后效性和最优子结构性质

问题结构分析

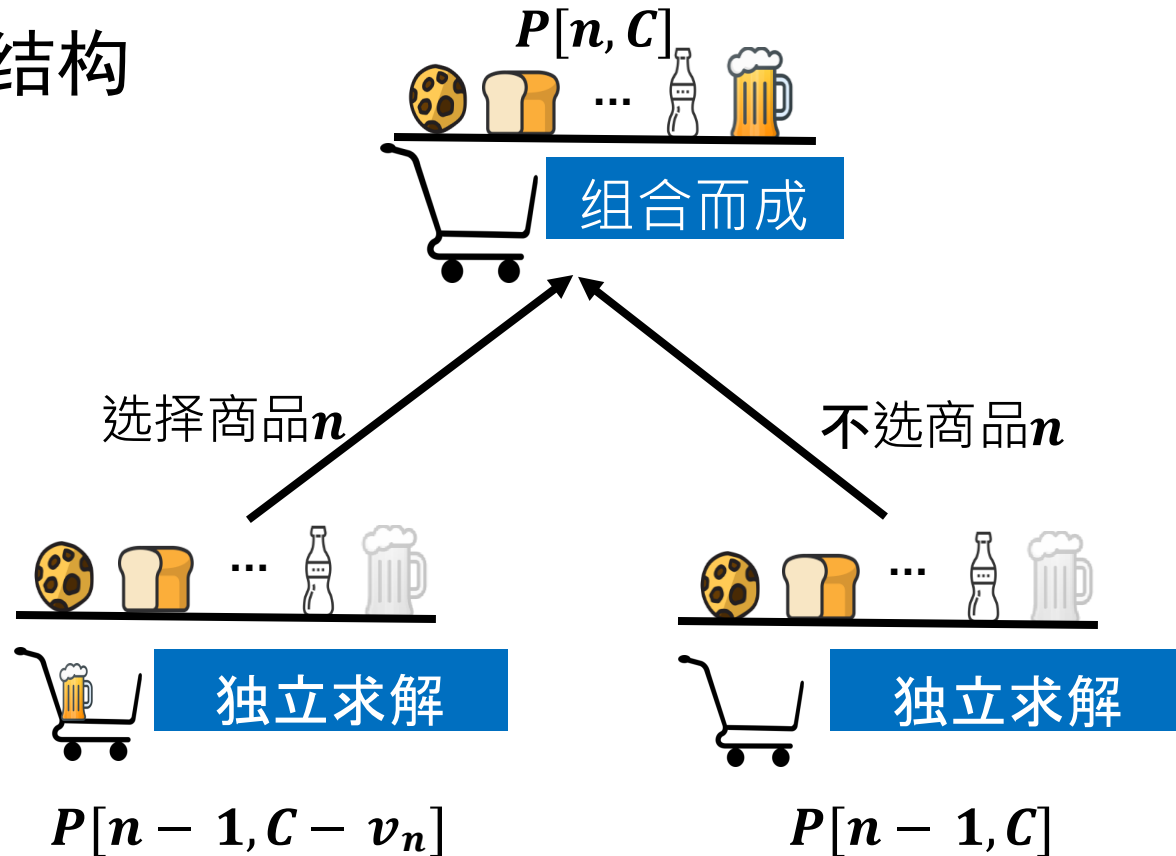
递推关系建立

自底向上计算

最优方案追踪

动态规划：一般步骤

- 分析最优（子）结构



- 构造递推公式

- $P[i, c] = \max\{P[i-1, c], P[i-1, c-v_i] + p_i\}$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：一般步骤

- 确定计算顺序
 - 初始化
 - $P[i, c]$ 依赖于子问题 $P[i - 1, c - v_i]$ 和 $P[i - 1, c]$
- 依次求解问题

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
2	0								
3	0								
4	0								
5	0								

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 记录决策过程

- $$Rec[i, c] = \begin{cases} 1, & \text{选择商品} \\ 0, & \text{不选商品} \end{cases}$$

$P[i, c]$	$c = 0$	1	2	3	...	10	11	12	13
$i = 0$	0	0	0	0	...	0	0	0	0
1	0								
...	0								
n	0								

Diagram illustrating the decision process in dynamic programming:

- A vertical arrow points from the cell $P[i, c]$ (where $i=10, c=10$) to the text $Rec[i, c] = 0$.
- A diagonal arrow points from the text $Rec[i, c] = 1$ to the cell $P[i, c]$ (where $i=10, c=10$).

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 输出最优方案

- $Rec[i, c] = 1$ 时, 选择商品 i , 考察子问题 $P[i - 1, c - v_i]$
- $Rec[i, c] = 0$ 时, 不选商品 i , 考察子问题 $P[i - 1, c]$

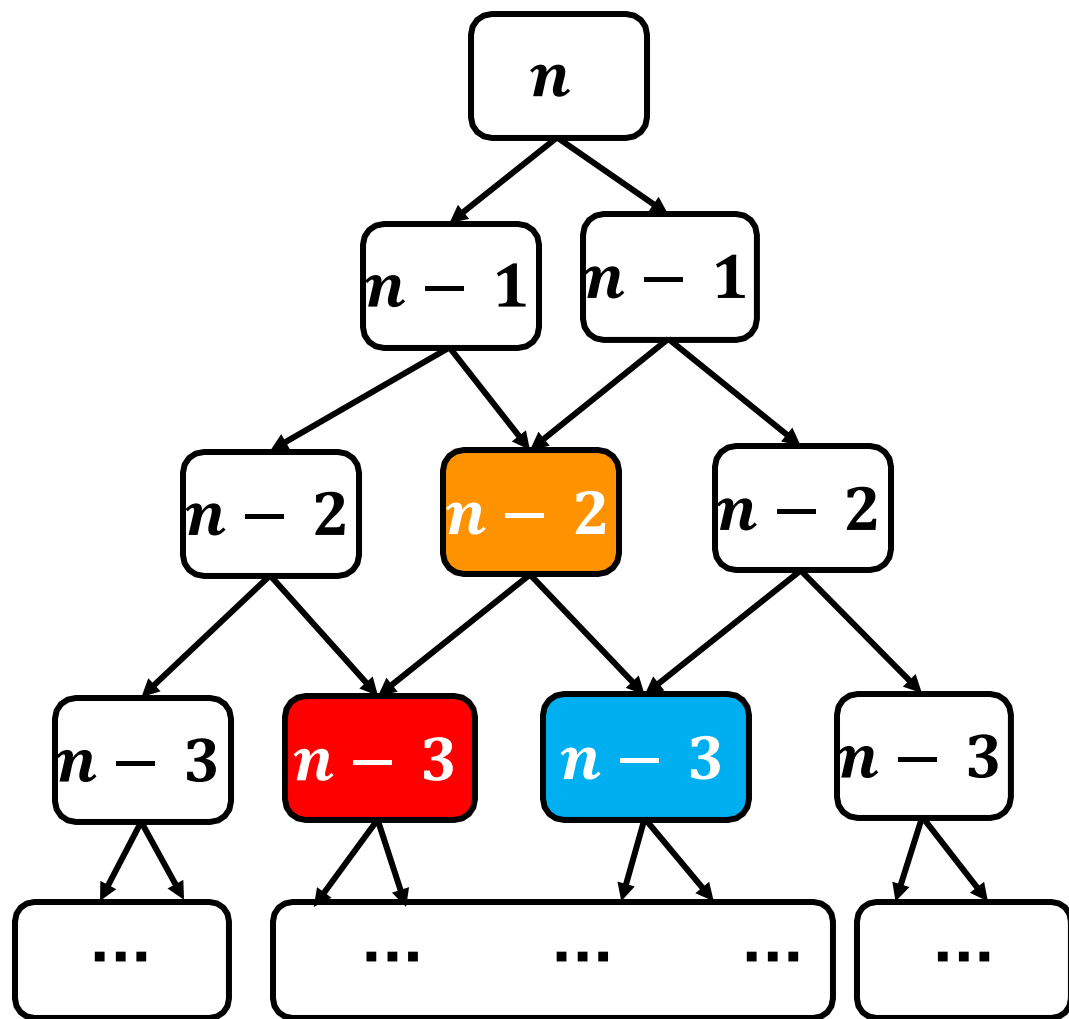
P	$c = 0$	1	2	...	9	10	11	12	13
$i = 1$									
2									
3									
4									
5									

问题结构分析

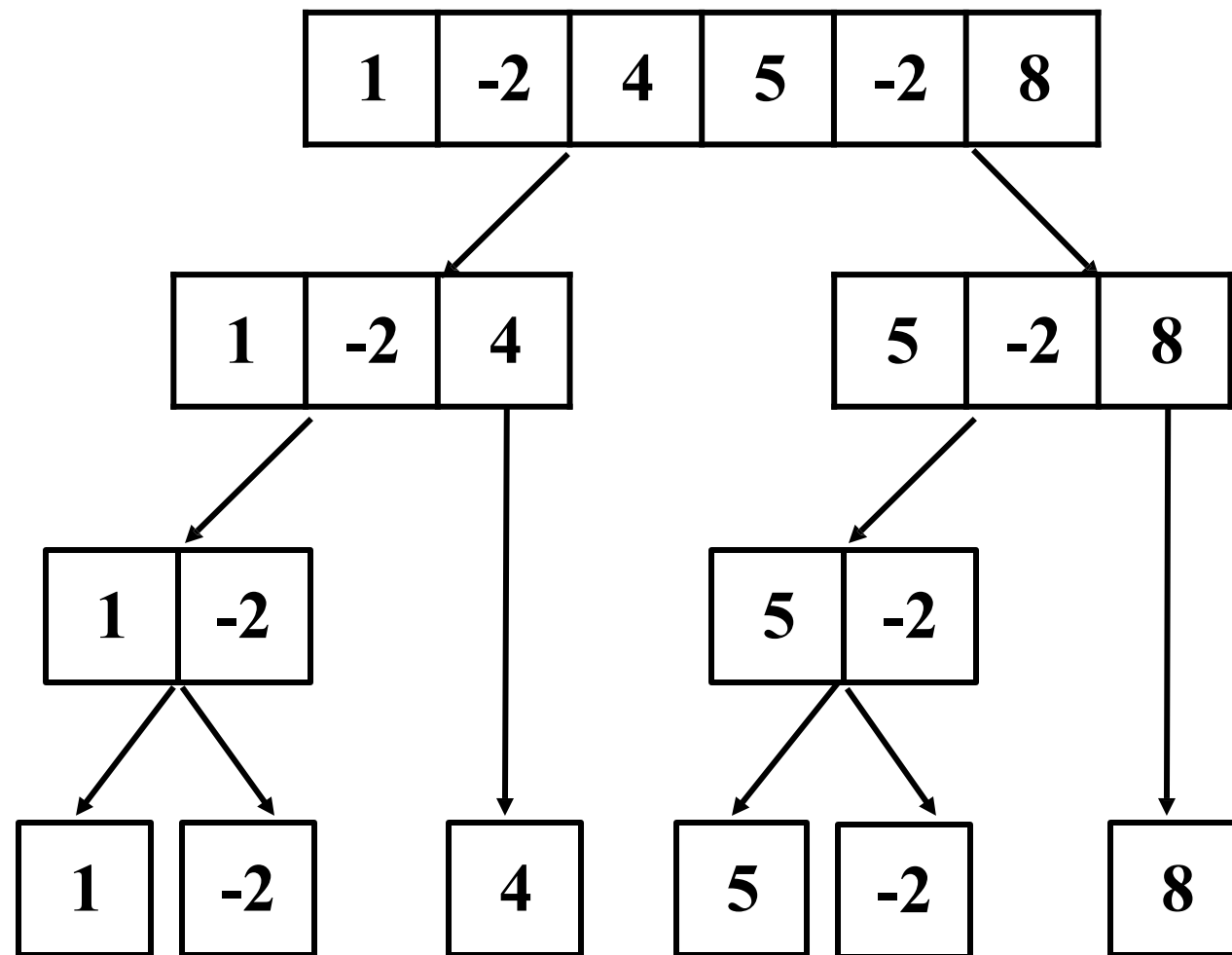
递推关系建立

自底向上计算

最优方案追踪



动态规划：重叠子问题



分而治之：独立子问题

动态规划：一般步骤

● 如何设计一个动态规划算法？四个步骤

● 问题结构分析

- 。给出问题表示，明确原始问题

● 递推关系建立

- 。分析最优结构，构造递推公式

● 自底向上计算

$$P[i, c] = \max\{P[i-1, c-v[i]] + p[i], P[i-1, c]\}$$

- 。确定计算顺序，依次求解问题

● 最优方案追踪

- 。记录决策过程，输出最优方案

思考：动态规划有什么缺点？

- 如果子问题重叠程度低，动态规划的时间复杂度改进不明显
- 过大的空间需求成为动态规划算法效率低下的主要因素

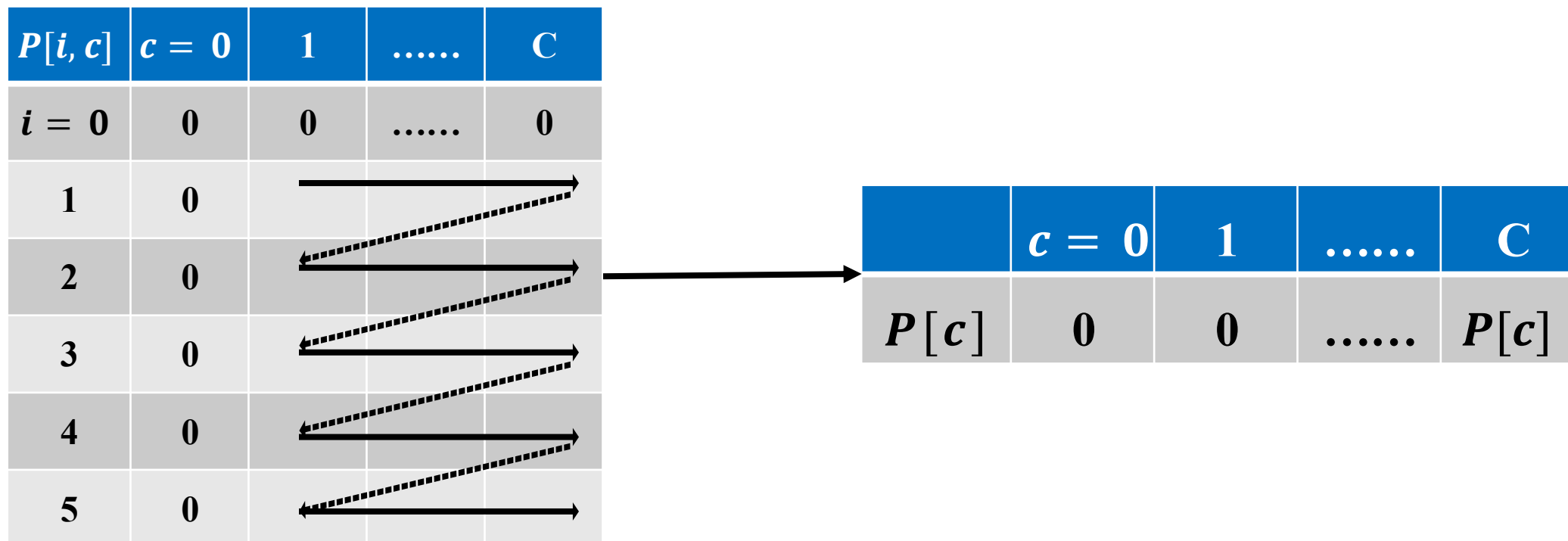
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- **5.3 再论背包问题**
- 5.4 可靠性设计
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题



$$P[i, c] = \max \left\{ \begin{array}{l} P[i-1, c-v[i]] + p[i], \\ P[i-1, c] \end{array} \right\} \longrightarrow P[c] = \max \{ P[c-v[i]] + p[i], P[c] \}$$

$$P[c] = \max\{P[c - v[i]] + p[i], P[c]\}$$

```
for i ← 0 to C do
```

```
    P[i] ← 0
```

```
for i ← 1 to n do
```

```
    for c ← v[i] to C do
```

```
        P[c] = max {P[c - v[i]] + p[i], P[c]}
```

思考：正确吗？

$$P[c] = \max\{P[c - v[i]] + p[i], P[c]\}$$

for $i \leftarrow 0$ to C do

$P[i] \leftarrow 0$

for $i \leftarrow 1$ to n do

 for $c \leftarrow C$ to $v[i]$ with step -1 do

$P[c] = \max\{P[c - v[i]] + p[i], P[c]\}$

反向递推！

思考：如何保存最优序列？

完全背包问题

- 输入：
 - n 个商品组成的集合 O ，每个商品有两个属性 v_i 和 p_i ，分别表示体积和价格
 - 背包容量 C
- 输出：
 - 求解放入背包的商品个数 $x_1, x_2, \dots, x_n \in \mathbb{N}$, 令

$$\max \sum_{i \in S} x_i p_i$$

优化目标

$$s. t. \sum_{i \in S} x_i v_i \leq C$$

约束条件

- 递推公式: $P[i, c] = \max_{k=0}^{+\infty} \{P[i-1, c-kv_i] + kp_i\}$
- 重用备忘录: $P[c] = \max\{P[c-v[i]] + p[i], P[c]\}$

思考：需要修改递推方向吗？

- 递推公式: $P[i, c] = \max_{k=0}^{+\infty} \{P[i-1, c-kv_i] + kp_i\}$
- 重用备忘录: $P[c] = \max\{P[c-v[i]] + p[i], P[c]\}$

```
for i ← 0 to C do
    P[i] ← 0
for i ← 1 to n do
    for c ← v[i] to C do
        P[c] = max {P[c - v[i]] + p[i], P[c]}
```

- 输入：
 - n 种商品组成的集合 O ，每种商品的个数为 k_i ，有两个属性 v_i 和 p_i ，分别表示体积和价格
 - 背包容量 C
- 输出：
 - 求解放入背包的商品个数 $x_1, x_2, \dots, x_n \in \mathbb{N}$, 令

$$\max \sum_{i \in S} x_i p_i$$

优化目标

$$s.t. \sum_{i \in S} x_i v_i \leq C \text{ and } x_i \leq k_i$$

约束条件

- 递推公式： $P[i, c] = \max_{k=0}^{k_i} \{P[i-1, c - kv_i] + kp_i\}$

0-1 背包递推函数

- $P[0, c] = \begin{cases} -\infty & c < 0 \\ 0 & c \geq 0 \end{cases}$

思考：若 $v[i]$, C 不是整数？

- $P[i, c] = \max\{P[i-1, c-v[i]] + p[i], P[i-1, c]\}$

- 令 $f_i(x) = P[i, x]$, 则

$$f_0(x) = \begin{cases} -\infty & x < 0 \\ 0 & x \geq 0 \end{cases}$$

$$f_i(x) = \max\{f_{i-1}(x-v[i]) + p[i], f_{i-1}(x)\}$$

- 把 $f_i(x)$ 看作一个函数

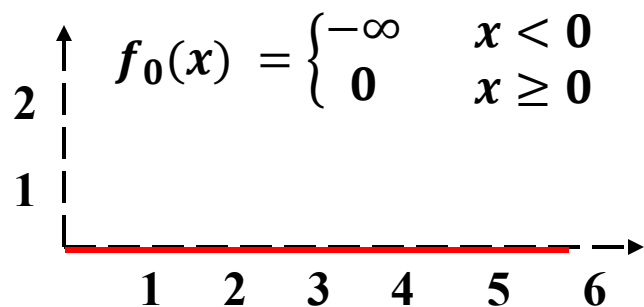
- 那么 $f_{i-1}(x-v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在 x 轴上右移 $v[i]$ 个单位, 再上移 $p[i]$ 个单位得到

- $f_i(x)$ 是 $f_{i-1}(x-v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成

0-1背包递推函数

- 把 $f_i(x)$ 看作一个函数
 - 那么 $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在x轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到
 - $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按x轴取最大值归并而成

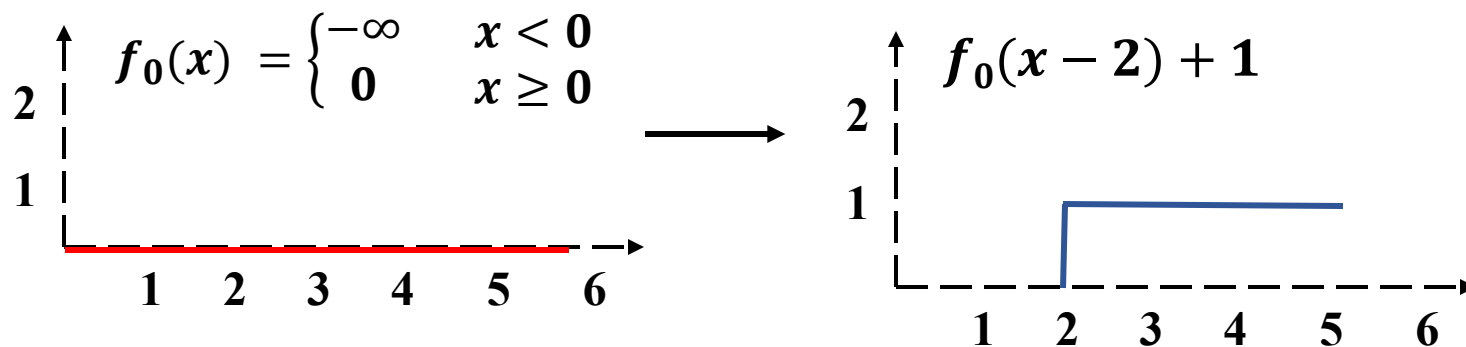
实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1, p_2, p_3)=(1,2,5)$, $C=6$



0-1背包递推函数

- 把 $f_i(x)$ 看作一个函数
 - 那么 $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在x轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到
 - $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按x轴取最大值归并而成

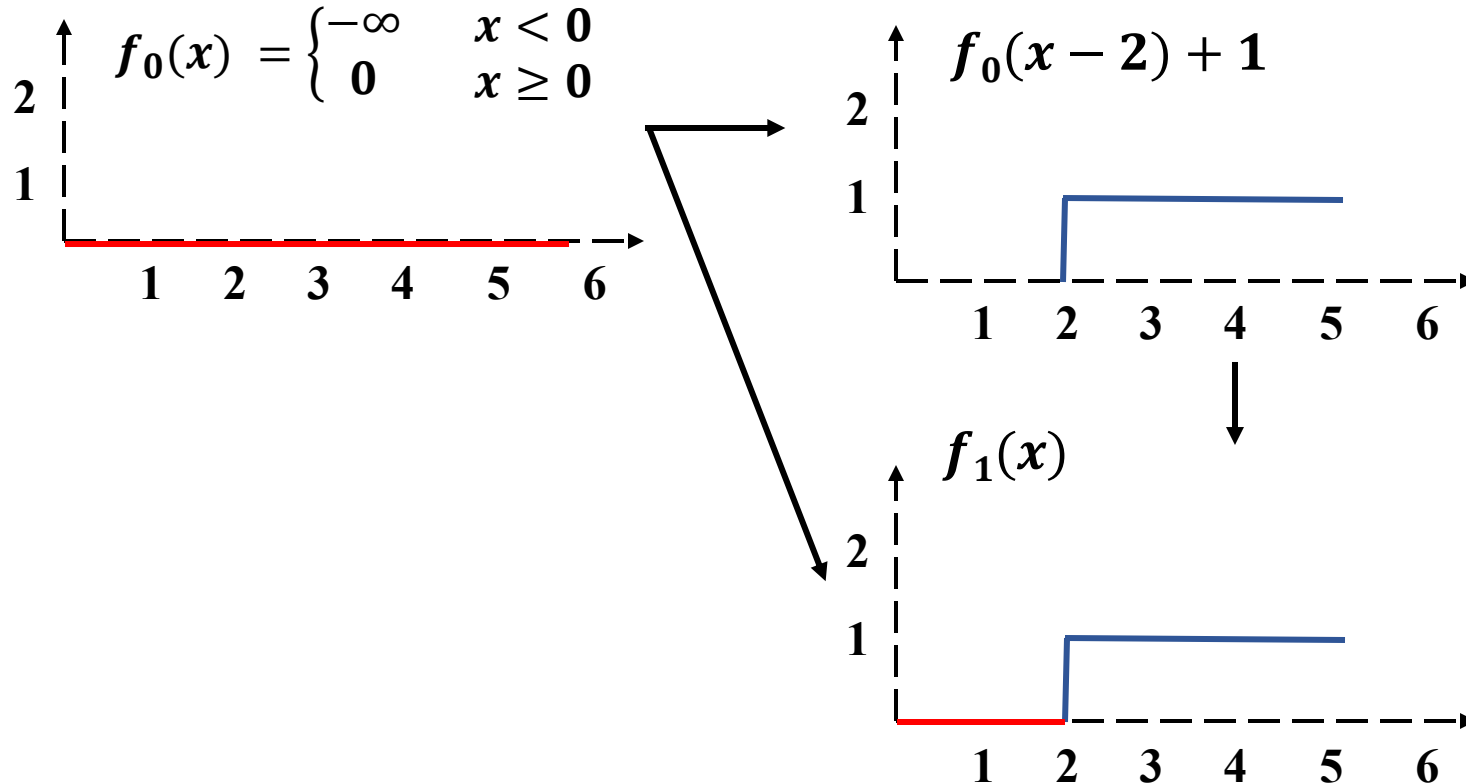
实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1, p_2, p_3)=(1,2,5)$, $C=6$



0-1背包递推函数

- 把 $f_i(x)$ 看作一个函数
 - 那么 $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在 x 轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到
 - $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成

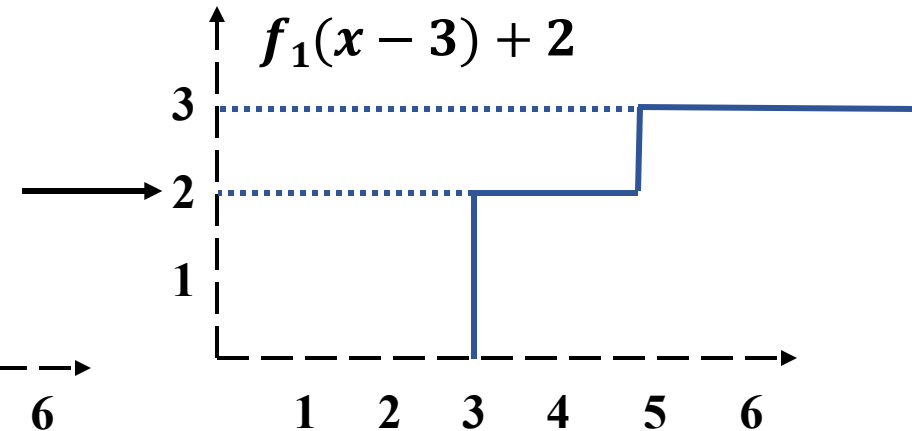
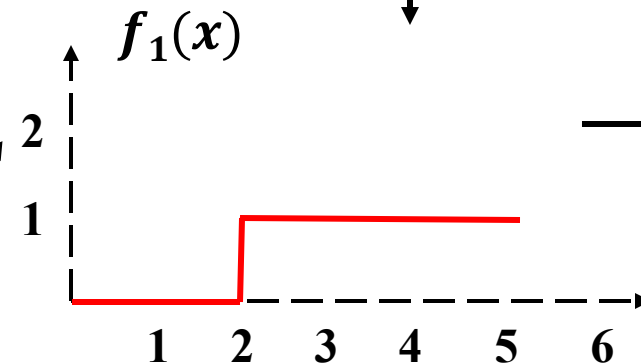
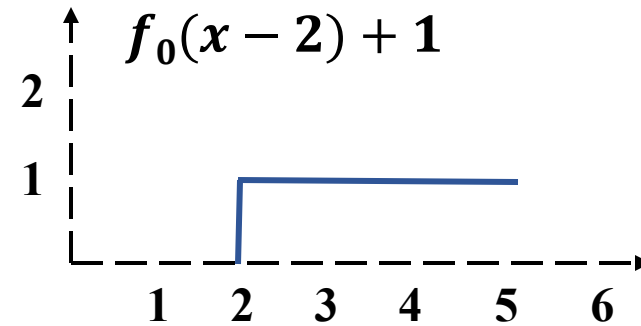
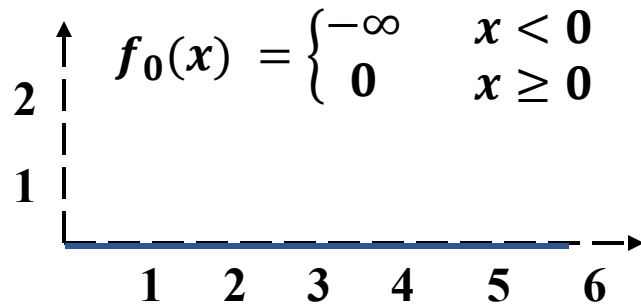
实例: $n=3, (v_1, v_2, v_3)=(2,3,4), (p_1, p_2, p_3)=(1,2,5), C=6$



0-1 背包递推函数

- 把 $f_i(x)$ 看作一个函数
 - 那么 $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在 x 轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到
 - $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成

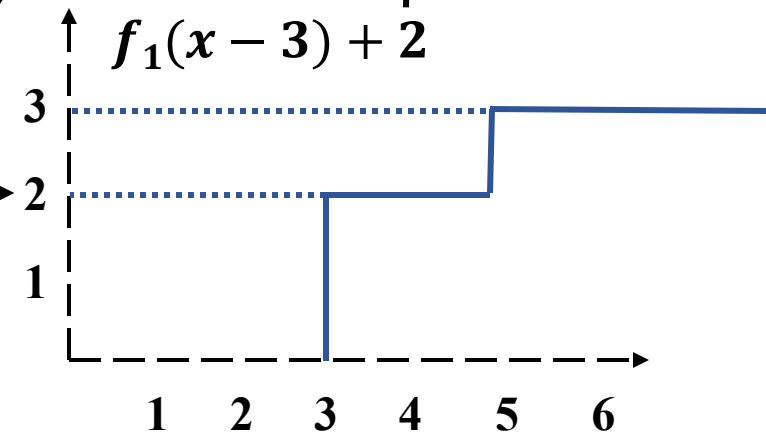
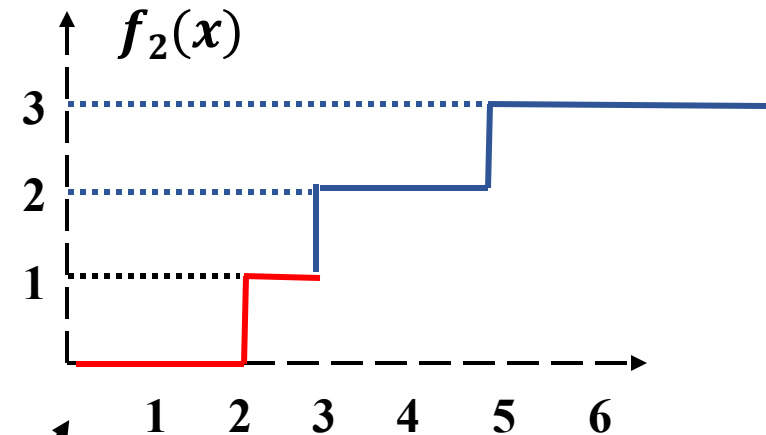
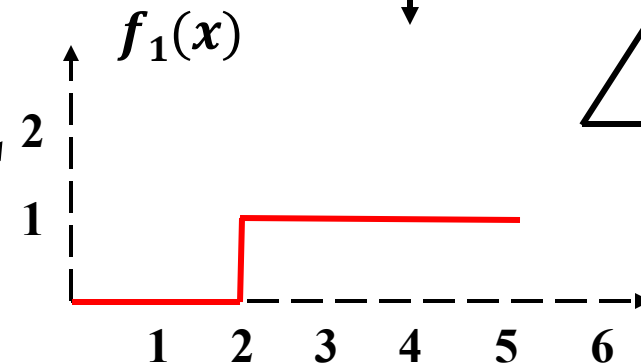
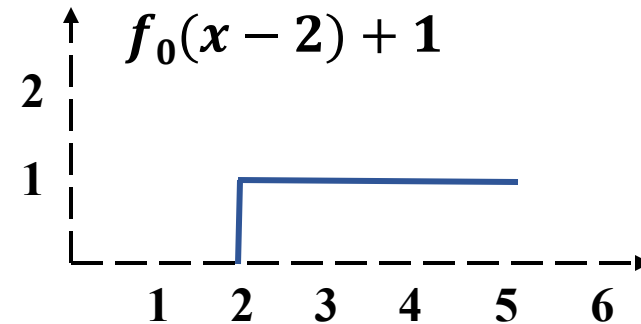
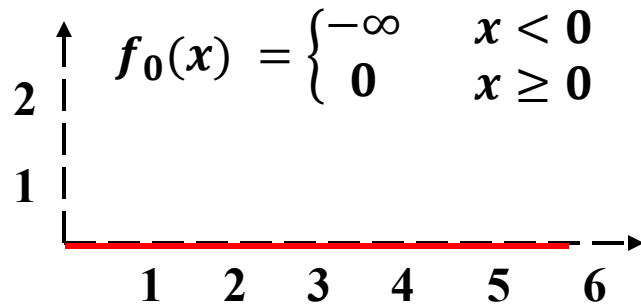
实例: $n=3, (v_1, v_2, v_3)=(2,3,4), (p_1, p_2, p_3)=(1,2,5), C=6$



0-1背包递推函数

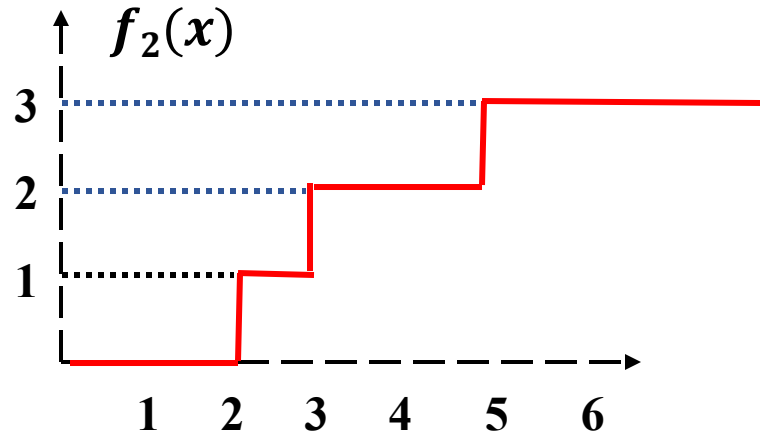
- 把 $f_i(x)$ 看作一个函数
 - 那么 $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在 x 轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到
 - $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成

实例: $n=3, (v_1, v_2, v_3)=(2,3,4), (p_1, p_2, p_3)=(1,2,5), C=6$



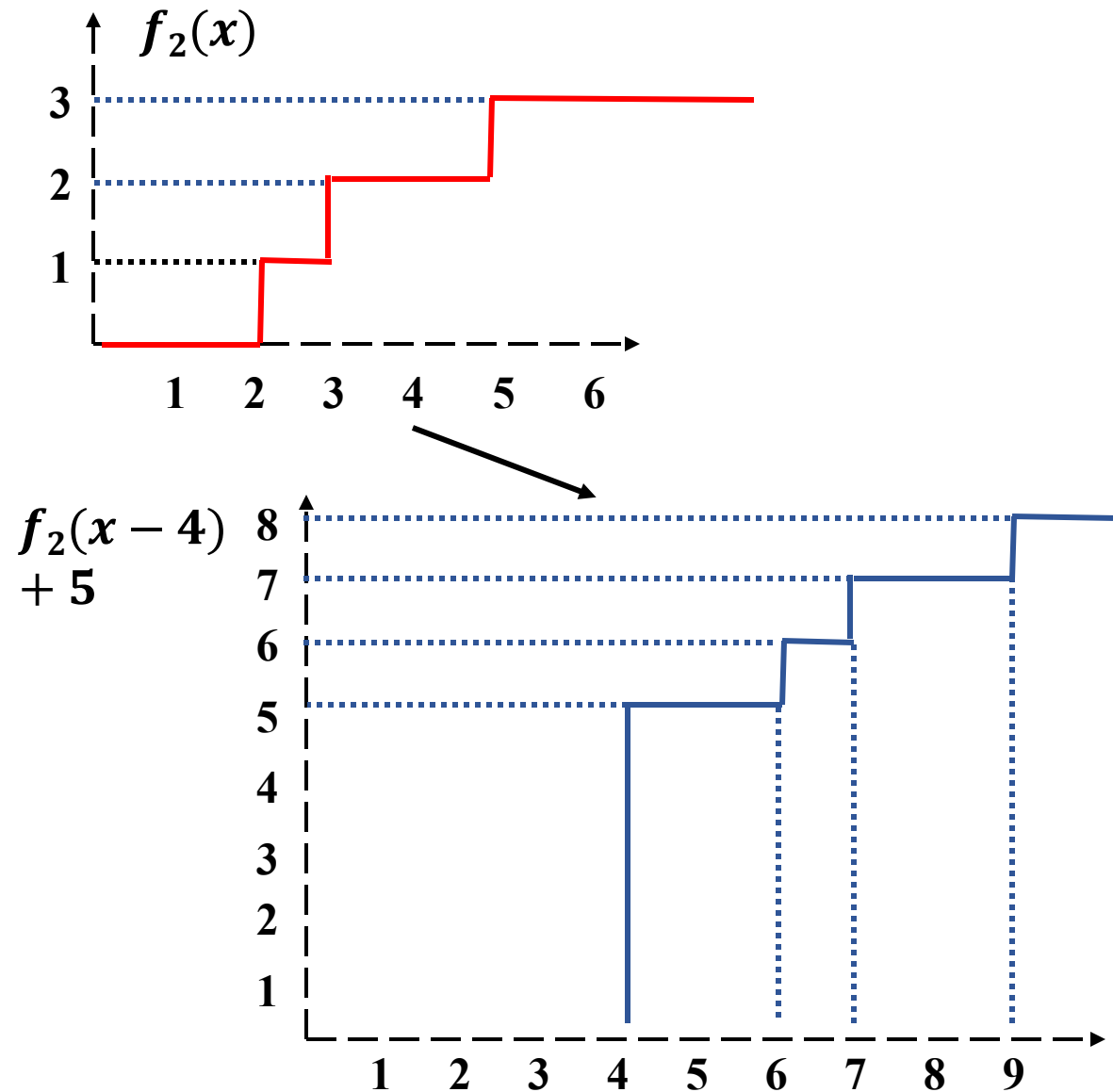
0-1 背包递推函数

实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1,p_2,p_3)=(1,2,5)$, $C=6$



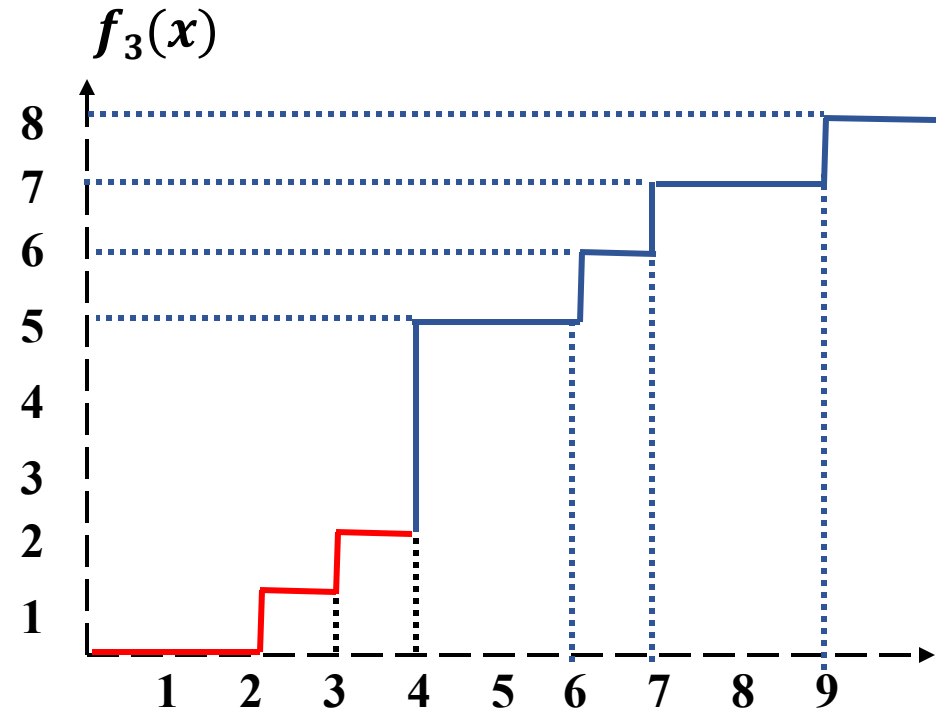
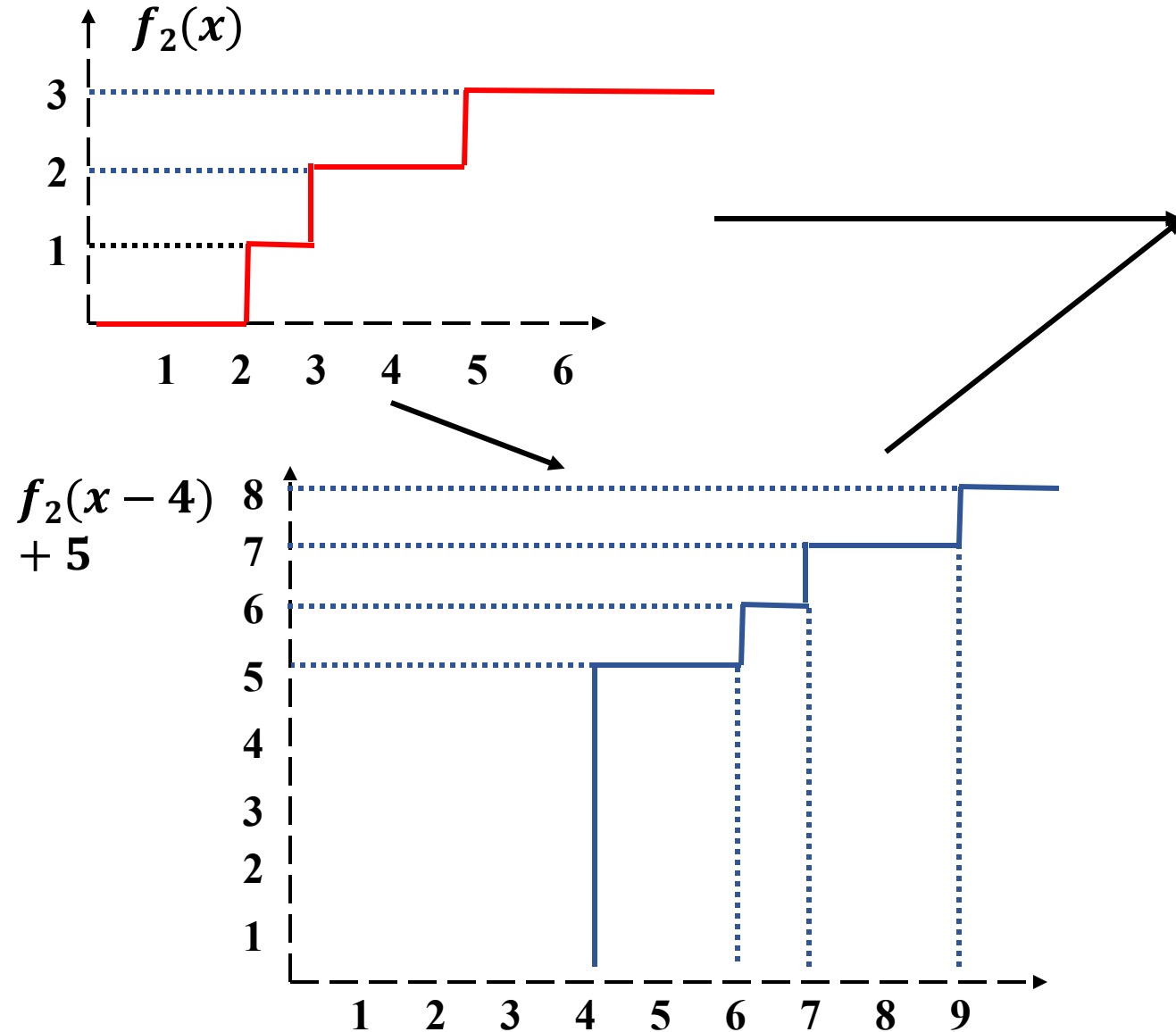
0-1 背包递推函数

实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1,p_2,p_3)=(1,2,5)$, $C=6$



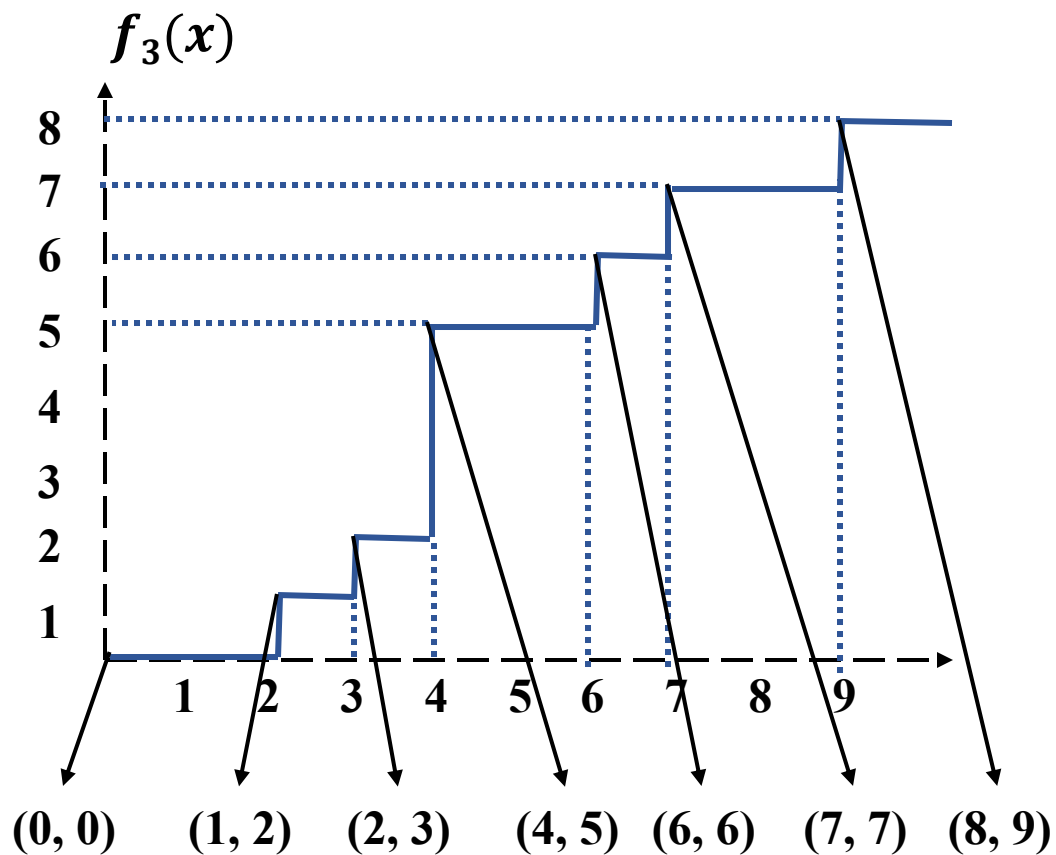
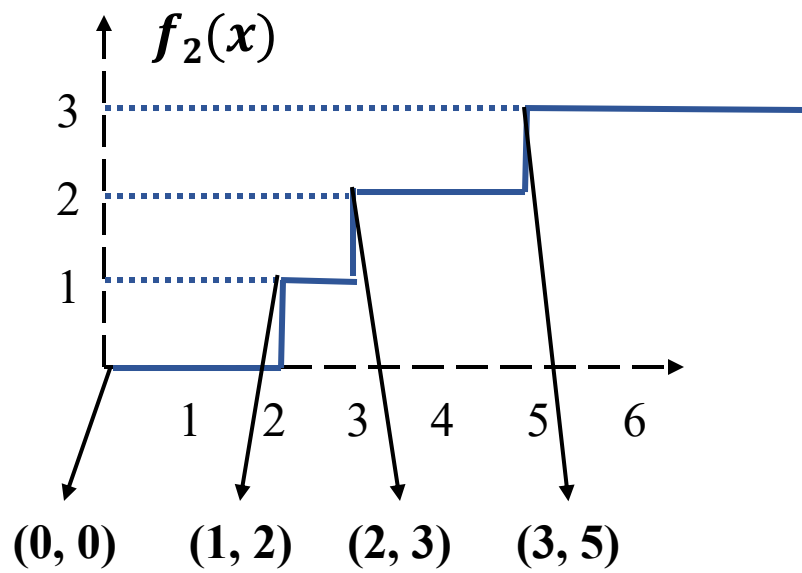
0-1 背包递推函数

实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1, p_2, p_3)=(1,2,5)$, $C=6$



函数的序偶表示

- 使用序偶集合表示函数曲线: (P, V) 表示在 $x=V$ 处函数阶跃到 P



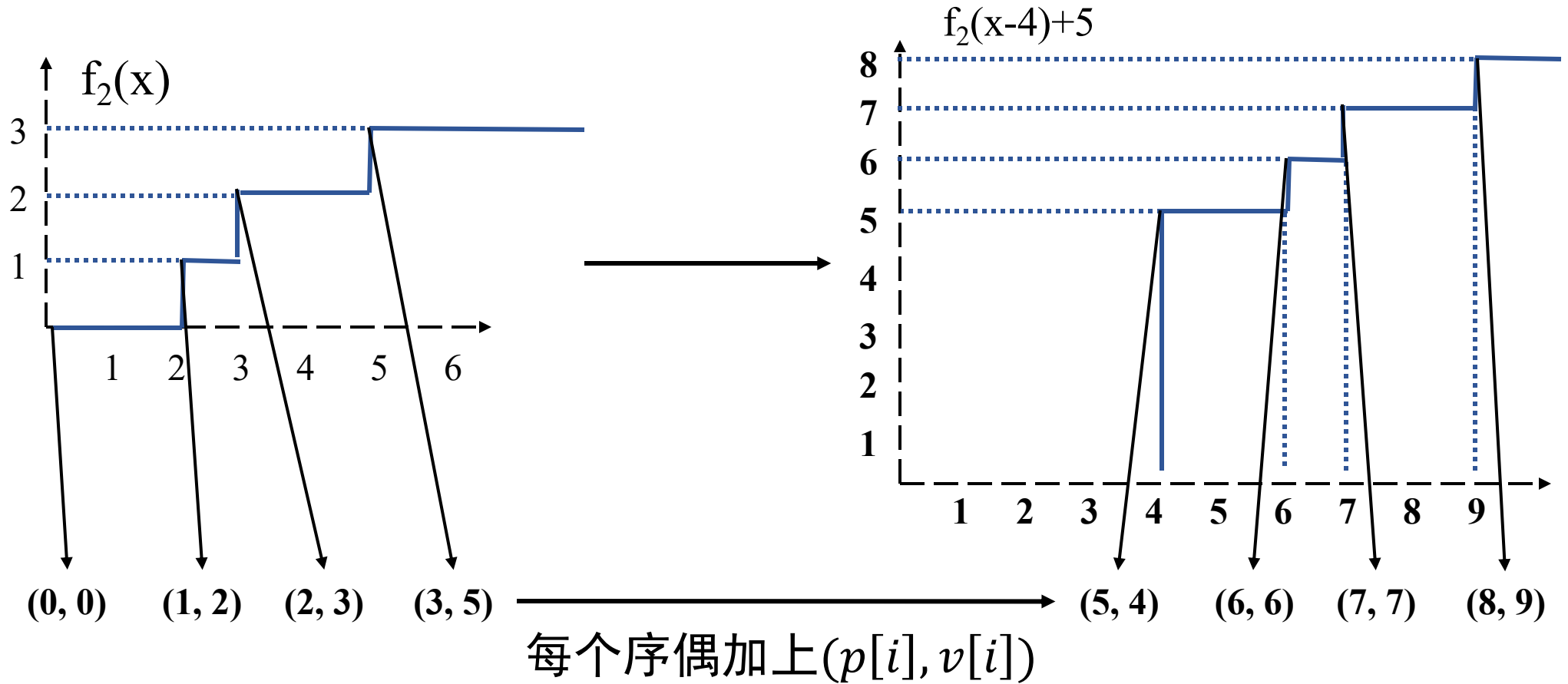
函数的序偶表示

- 使用序偶集合表示函数曲线: (P, V) 表示在 $x=V$ 处函数阶跃到 P
- 假设函数被表示成 r 个序偶 $(P_1, V_1), (P_2, V_2), \dots, (P_r, V_r)$
- 对所有满足 $V_j \leq x < V_{j+1}$ 的 x , 有 $f_i(x) = P_j$; 对所有满足 $x \geq V_r$ 的 x , 有 $f_i(x) = P_r$
- 最优解等于 P_r

思考：如何递推函数的序偶表示？

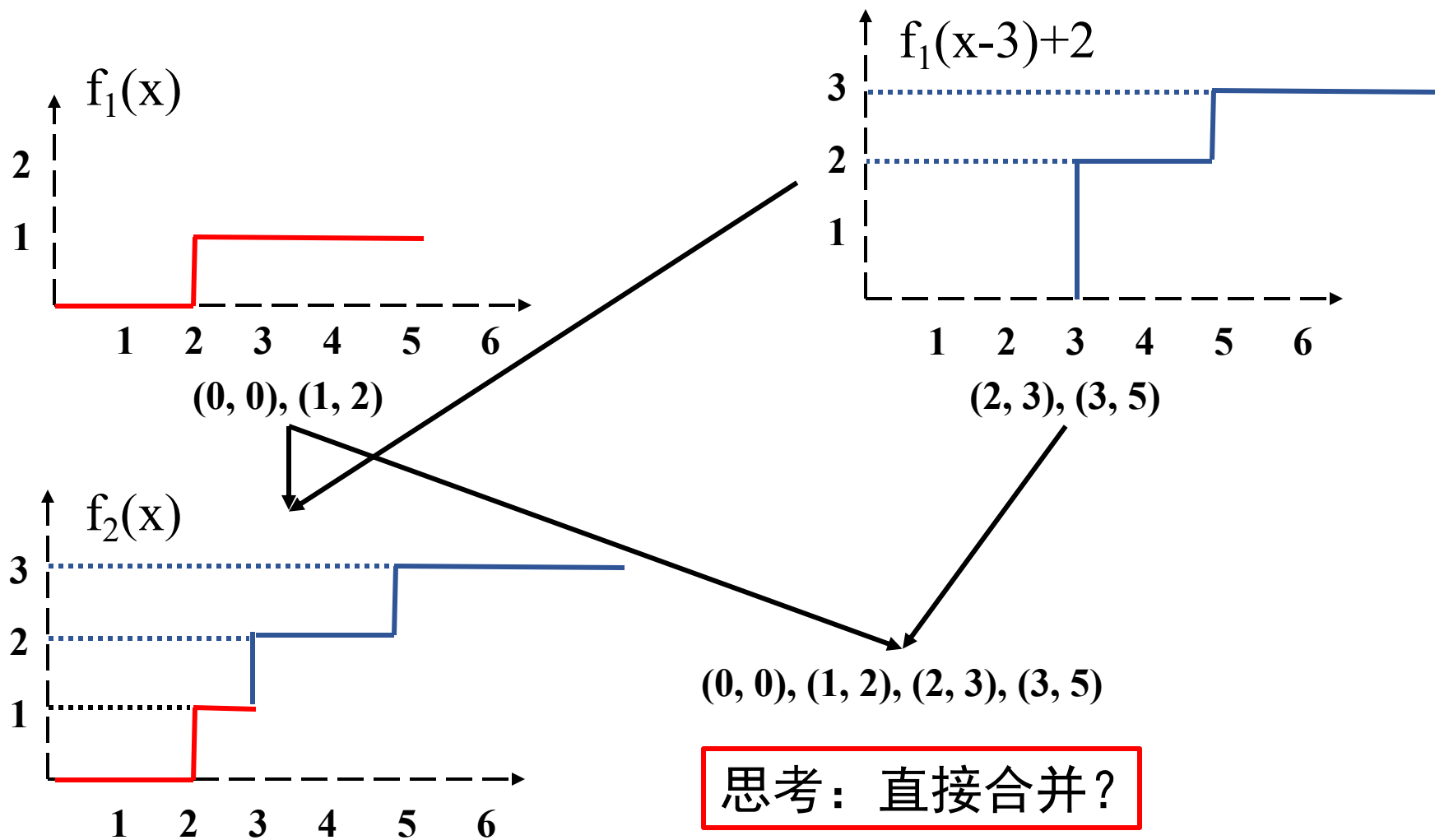
函数的序偶递推

- $f_{i-1}(x - v[i]) + p[i]$ 是 $f_{i-1}(x)$ 在 x 轴上右移 $v[i]$ 个单位，再上移 $p[i]$ 个单位得到



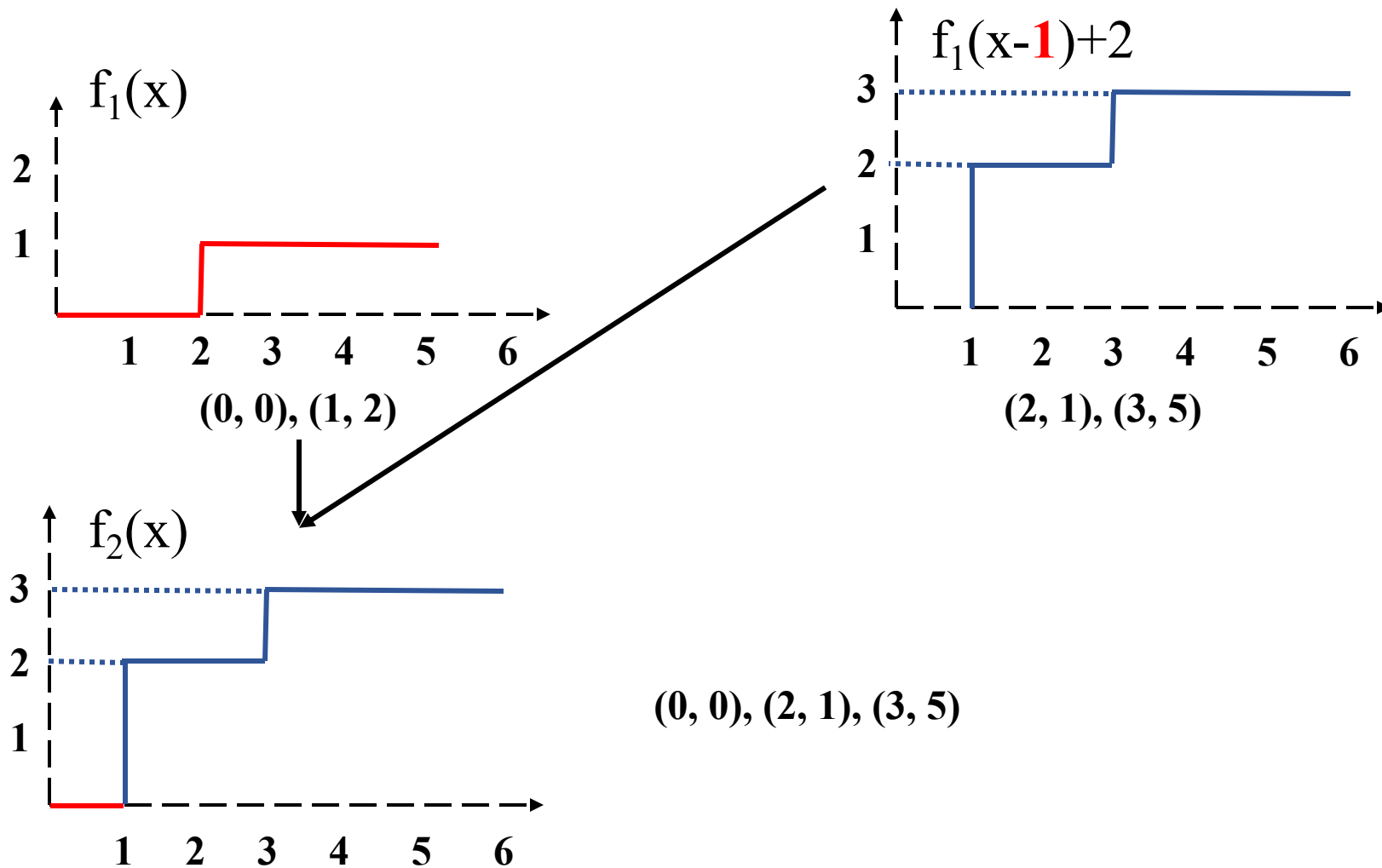
函数的序偶递推

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成



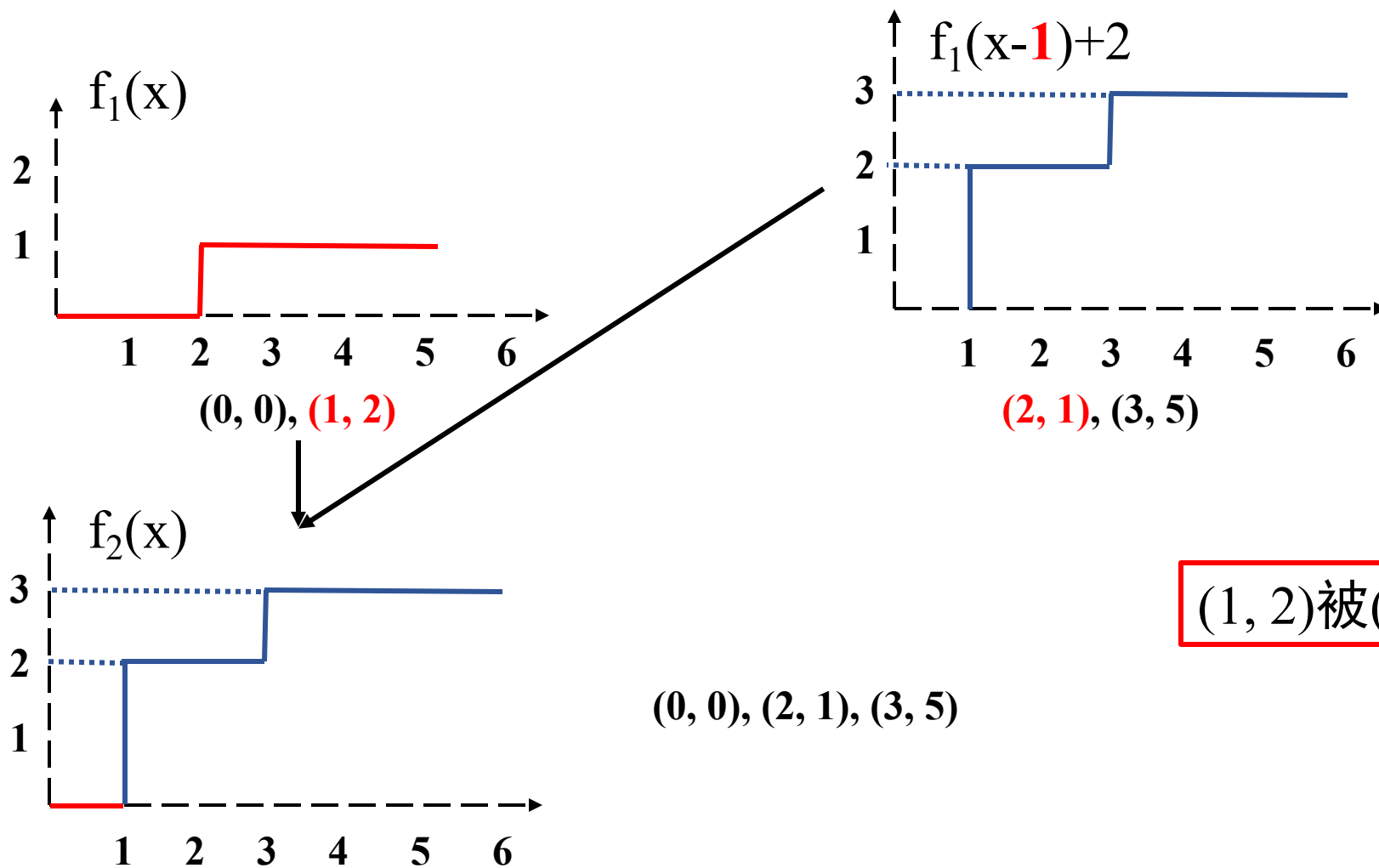
函数的序偶递推

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成



函数的序偶递推

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成



(1, 2) 被 (2, 1) 支配, 舍去!

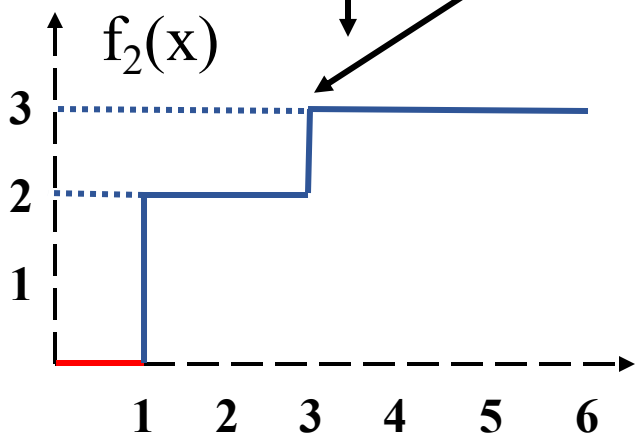
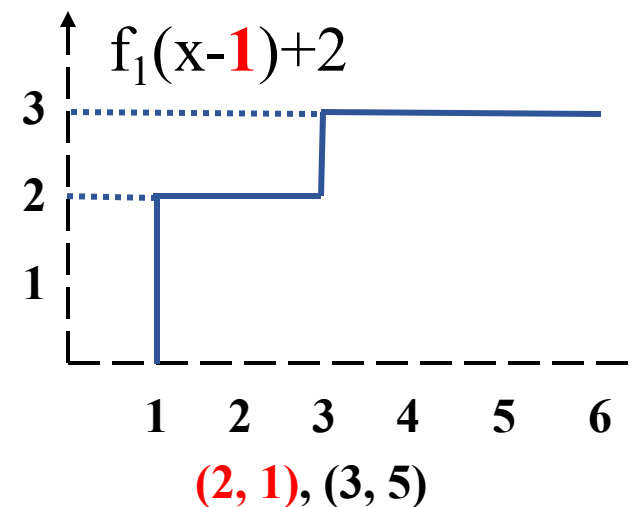
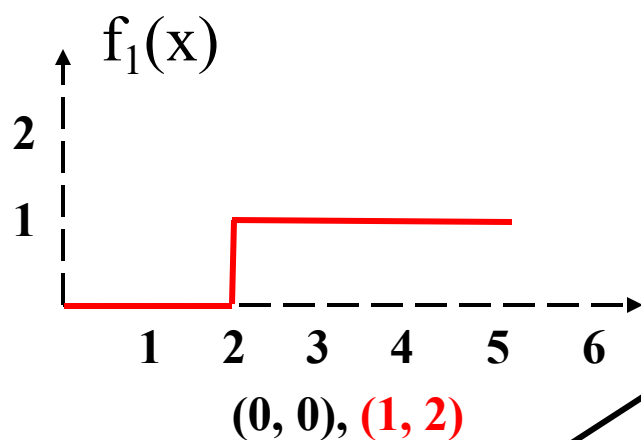
函数的序偶递推

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成
- S^{i-1} : f_{i-1} 的所有序偶的集合
- S_1^i : $f_{i-1}(x-v_i)+p_i$ 的所有序偶的集合

支配规则: 对于任意 $(P_j, V_j) \in S^{i-1}$, 若存在 $(P_k, V_k) \in S_1^i$, 使得 $V_j \geq V_k$ 的同时有 $P_j \leq P_k$, 那么序偶 (P_j, V_j) 被放弃, 称为 (P_k, V_k) 支配 (P_j, V_j) 。

函数的序偶递推

支配规则: 对于任意 $(P_j, V_j) \in S^{i-1}$, 若存在 $(P_k, V_k) \in S^i_1$, 使得 $V_j \geq V_k$ 的同时有 $P_j \leq P_k$, 那么序偶 (P_j, V_j) 被放弃, 称为 (P_k, V_k) 支配 (P_j, V_j) 。



$(0, 0), (2, 1), (3, 5)$

- $2 \geq 1$ 且 $1 \leq 2$
- $(1, 2)$ 被 $(2, 1)$ 支配, 舍去!

函数的序偶递推

- $S^0 = \{(0, 0)\}$
- $i \geq 1$ 时, 将 (p_i, v_i) 加到 S^{i-1} 的每一对序偶上就得到 S_1^i
即 $S_1^i = \{(P + p_i, V + v_i) | (P, V) \in S^{i-1}\}$
- 在 **支配规则** 下将 S^{i-1} 和 S_1^i 归并成 S^i
- 在生成 S^i 时, 将 $V > C$ 的那些序偶 (P, V) 去掉, 它们不能导出满足约束条件的可行解

S^i 中的序偶对表示对于不同的约束 $V \leq C$, 前 i 个物品可能取得的最大效益值。

实例分析

实例: $n=3$, $(v_1, v_2, v_3)=(2,3,4)$, $(p_1, p_2, p_3)=(1,2,5)$, $C=6$

$$S^0 = \{(0,0)\} \xrightarrow{(p_1, v_1)=(1,2)} S^1_1 = \{(1,2)\}$$

$$S^1 = \{(0,0), (1,2)\} \xrightarrow{(p_2, v_2)=(2,3)} S^2_1 = \{(2,3), (3,5)\}$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\} \xrightarrow{(p_3, v_3)=(5,4)}$$

$$S^3_1 = \{(5,4), (6,6), (7,7), (8,9)\}$$

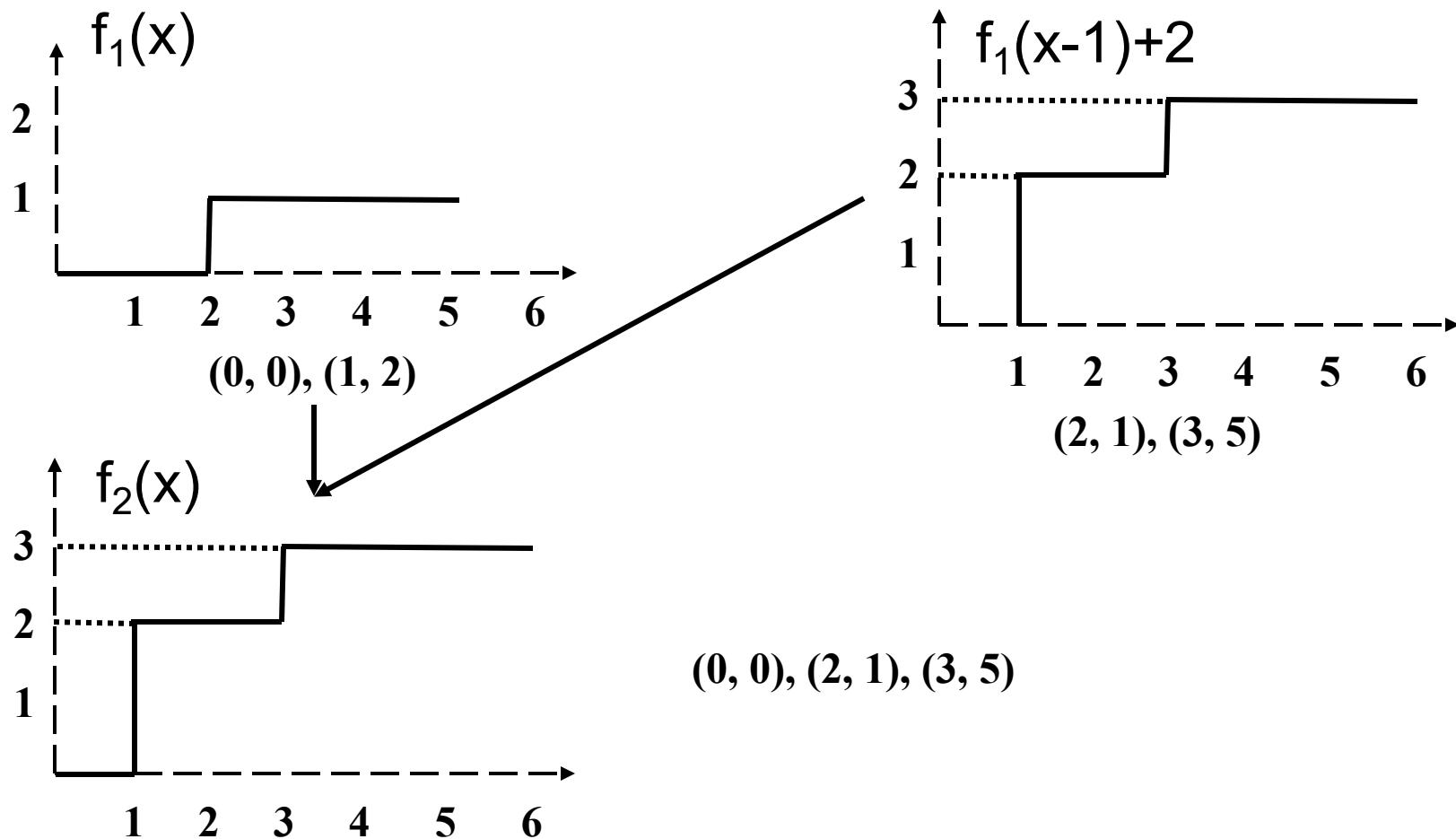
$V > C$, 舍弃掉

根据支配规则, $(5,4)$
支配掉 $(3,5)$

$$S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\} \text{ 最优解 } 6$$

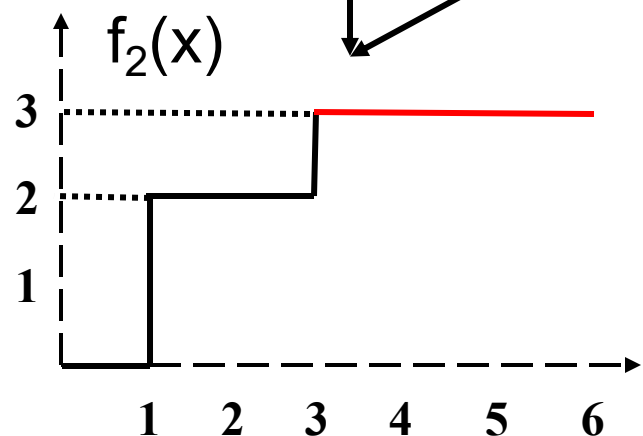
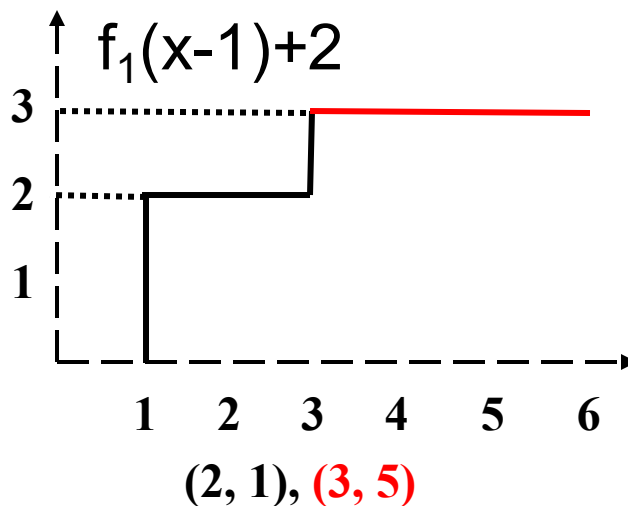
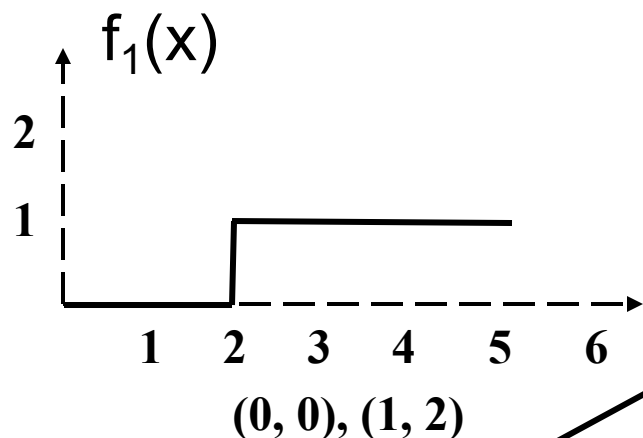
确定决策序列

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成
- 若 $f_i(x)$ 的最末序偶来自 $f_{i-1}(x - v[i]) + p[i]$, 则选取第 i 个物品
- 若 $f_i(x)$ 的最末序偶来自 $f_{i-1}(x)$, 则不选取第 i 个物品



确定决策序列

- $f_i(x)$ 是 $f_{i-1}(x - v[i]) + p[i]$ 和 $f_{i-1}(x)$ 按 x 轴取最大值归并而成
- 若 $f_i(x)$ 的最末序偶来自 $f_{i-1}(x - v[i]) + p[i]$, 则选取第 i 个物品
- 若 $f_i(x)$ 的最末序偶来自 $f_{i-1}(x)$, 则不选取第 i 个物品



- (3, 5) 来自 $f_1(x-1)+2$
- 选择第2个物品

- 对于 S^n 的最末序偶 (P_l, W_l) , 若 $(P_l, W_l) \in S^{n-1}$, 则 $x_n=0$; 若 $(P_l, W_l) \notin S^{n-1}$, 则 $x_n=1$
- 再判断留在 S^{n-1} 的序偶 (P_l, W_l) 或 $(P_l - p_n, W_l - w_n)$ 是否属于 S^{n-2} 以确定 x_{n-1} 的取值
- ...

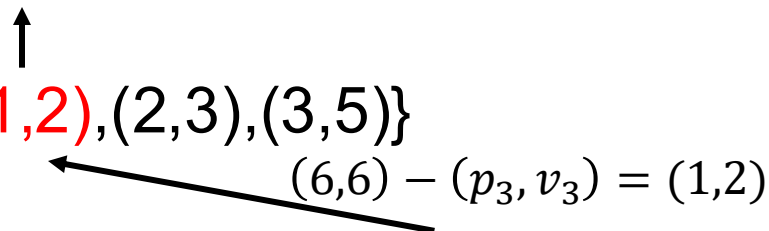
实例: $n=3, (v_1, v_2, v_3)=(2,3,4), (p_1, p_2, p_3)=(1,2,5), C=6$

$$S^0 = \{(0,0)\}$$

$$S^1 = \{(0,0), (1,2)\} \quad x_1=1$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\} \quad x_2=0$$

$$S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\} \quad x_3=1$$



最优决策序列是 $(x_1, x_2, x_3)=(1,0,1)$

0-1背包问题的序偶对算法

输入：商品数量 n ，各商品价值 p_i ，体积 v_i ，背包容量 C

输出：商品价格的最大值，最优化方案

$S^0 \leftarrow \{(0, 0)\}$

初始化

for $i \leftarrow 1$ to $n - 1$ do

$S_1^i \leftarrow \{(P + p_i, V + v_i) \mid (P, V) \in S^{i-1} \text{ and } V \leq C\}$

$S^i \leftarrow \text{Merge-Purge}(S^{i-1}, S_1^i)$

$(P_{\max}, V_{\max}) \leftarrow S^i$ 最末序偶

for $i \leftarrow n$ to 1 do

if $(P_{\max}, V_{\max}) \in S^{i-1}$ then $x_i = 0$

else $x_i = 1$; $(P_{\max}, V_{\max}) \leftarrow (P_{\max}, V_{\max}) - (p_i, v_i)$ end

end

return S^n 最末序偶中的 P , x

0-1背包问题的序偶对算法

输入：商品数量 n ，各商品价值 p_i ，体积 v_i ，背包容量 C

输出：商品价格的最大值，最优化方案

$S^0 \leftarrow \{(0, 0)\}$

for $i \leftarrow 1$ to $n - 1$ do

$S^i_1 \leftarrow \{(P + p_i, V + v_i) \mid (P, V) \in S^{i-1} \text{ and } V \leq C\}$

$S^i \leftarrow \text{Merge-Purge}(S^{i-1}, S^i_1)$

$(P_{\max}, V_{\max}) \leftarrow S^i$ 最末序偶

for $i \leftarrow n$ to 1 do

if $(P_{\max}, V_{\max}) \in S^{i-1}$ then $x_i = 0$

else $x_i = 1$; $(P_{\max}, V_{\max}) \leftarrow (P_{\max}, V_{\max}) - (p_i, v_i)$ end

end

return S^n 最末序偶中的 P , x

递推序偶对

0-1背包问题的序偶对算法

输入：商品数量 n ，各商品价值 p_i ，体积 v_i ，背包容量 C

输出：商品价格的最大值，最优化方案

$S^0 \leftarrow \{(0, 0)\}$

for $i \leftarrow 1$ to $n - 1$ do

$S_1^i \leftarrow \{(P + p_i, V + v_i) \mid (P, V) \in S^{i-1} \text{ and } V \leq C\}$

$S^i \leftarrow \text{Merge-Purge}(S^{i-1}, S_1^i)$

$(P_{\max}, V_{\max}) \leftarrow S^i$ 最末序偶

for $i \leftarrow n$ to 1 do

if $(P_{\max}, V_{\max}) \in S^{i-1}$ then $x_i = 0$

else $x_i = 1$; $(P_{\max}, V_{\max}) \leftarrow (P_{\max}, V_{\max}) - (p_i, v_i)$ end

end

return S^n 最末序偶中的 P , x

确定决策序列

空间复杂度分析

输入：商品数量 n ，各商品价值 p_i ，体积 v_i ，背包容量 C

输出：商品价格的最大值，最优化方案

$S^0 \leftarrow \{(0, 0)\}$

for $i \leftarrow 1$ to $n - 1$ do

$S^i_1 \leftarrow \{(P + p_i, V + v_i) \mid (P, V) \in S^{i-1} \text{ and } V \leq C\}$

$S^i \leftarrow \text{Merge-Purge}(S^{i-1}, S^i_1)$

$(P_{\max}, V_{\max}) \leftarrow S^i$ 最末序偶

for $i \leftarrow n$ to 1 do

if $(P_{\max}, V_{\max}) \in S^{i-1}$ then $x_i = 0$

else $x_i = 1$; $(P_{\max}, V_{\max}) \leftarrow (P_{\max}, V_{\max}) - (p_i, v_i)$ end

end

return S^n 最末序偶中的 P , x

$$|S^i_1| \leq |S^{i-1}|$$

空间复杂度分析

输入：商品数量 n ，各商品价值 p_i ，体积 v_i ，背包容量 C

输出：商品价格的最大值，最优化方案

$S^0 \leftarrow \{(0, 0)\}$

for $i \leftarrow 1$ to $n - 1$ do

$S^i_1 \leftarrow \{(P + p_i, V + v_i) \mid (P, V) \in S^{i-1} \text{ and } V \leq C\}$

$S^i \leftarrow \text{Merge-Purge}(S^{i-1}, S^i_1)$

$$|S^i| \leq 2|S^{i-1}|$$

$(P_{\max}, V_{\max}) \leftarrow S^i$ 最末序偶

for $i \leftarrow n$ to 1 do

if $(P_{\max}, V_{\max}) \in S^{i-1}$ then $x_i = 0$

else $x_i = 1$; $(P_{\max}, V_{\max}) \leftarrow (P_{\max}, V_{\max}) - (p_i, v_i)$ end

end

return S^n 最末序偶中的 P ， x

- 最坏情况下，归并过程中没有序偶被清除，则
- $|S^0|=1$ ， $|S^1|=2$ ， $|S^2|=2+2=2^2$ ，.....， $|S^{n-1}|=2^{n-1}$
- 则空间复杂度是 $O(2^n)$

时间复杂度分析

- 由 S^{i-1} 生成 S^i ，需要的时间是 $\Theta(|S^{i-1}|)$
- 计算所有的 $S^0, S^1, \dots, S^{n-1}$ ，需要的总时间为 $\Theta(\sum |S^{i-1}|)$
- 因为 $|S^i| \leq 2^i$ ，故总时间为 $O(2^n)$
- 若 p, v 均为整数，则有 $|S^i| \leq 1 + \sum_{1 \leq j \leq i} p_j$ ， $|S^i| \leq 1 + \min \{ \sum_{1 \leq j \leq i} v_j, C \}$
- 时间复杂度为

$$O(\min \{ 2^n, n \sum_{1 \leq j \leq n} p_j, nC \})$$

C 一般远小于 2^n ，再加上支配规则，能解决 n 较大时的问题

思考：比较迭代法和序偶对法的时间复杂度？

• 试探法

- L:最优解的估计值, $f_n(C) \geq L$
- $PLEFT(i) = \sum_{i < j \leq n} p_j$, 如果 $(P, V) \in S^i$, 且 $P + PLEFT(i) < L$, 那么 (P, V) 从 S^i 中清除掉

实例: $n=6$, $C=165$

$$\begin{aligned} & (p_1, p_2, p_3, p_4, p_5, p_6) \\ &= (v_1, v_2, v_3, v_4, v_5, v_6) \\ &= (100, 50, 20, 10, 7, 3), \end{aligned}$$

	0	1	2	3	4	5	6
PLEFT(i)	190	90	40	20	10	3	0

• 贪心方法求出估计值 $L=163$

• $S^0 = \{0\}$, $S^1_1 = \{100\}$

• $S^1 = \{\text{0}, 100\}$, $S^2_1 = \{150\}$

• $S^2 = \{\text{+100}, 150\}$, $S^3_1 = \emptyset$

• $S^3 = \{150\}$, $S^4_1 = \{160\}$

• $S^4 = \{\text{+150}, 160\}$, $S^5_1 = \emptyset$

• $S^5 = \{160\}$, $S^6_1 = \{163\}$

• $S^6 = \{163\}$

确定为最优值!